

\$\$\$ sql.banco

```
DROP DATABASE IF EXISTS banco;
```

```
CREATE DATABASE banco CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

```
USE banco;
```

```
CREATE TABLE seguradoras (  
    nome VARCHAR(50) PRIMARY KEY,  
    cidade VARCHAR(50),  
    cobertura_percentual INT,  
    possui_atendimento_24h BOOLEAN,  
    forma_pagamento_preferencial VARCHAR(20)  
);
```

```
CREATE TABLE sinistros (  
    id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,  
    segurado VARCHAR(80) NOT NULL,  
    telefone VARCHAR(20),  
    cidade VARCHAR(50),  
    grau_monta VARCHAR(20),  
    perda_total BOOLEAN  
);
```

```
CREATE TABLE pecas (  
    codigo INT PRIMARY KEY,  
    nome VARCHAR(50),  
    marca VARCHAR(20),  
    preco DECIMAL(10,2),  
    mao_obra_propria BOOLEAN,  
    dias_garantia INT,  
    cor VARCHAR(20)  
);
```

```
CREATE TABLE pecas_sinistros (  
    peca_codigo INT NOT NULL,  
    sinistro_id INT NOT NULL,
```

```
PRIMARY KEY (peca_codigo, sinistro_id),  
FOREIGN KEY (peca_codigo) REFERENCES pecas(codigo) ON DELETE CASCADE,  
FOREIGN KEY (sinistro_id) REFERENCES sinistros(id) ON DELETE CASCADE  
);
```

\$\$\$ src.controles.ControladorCadastroPecas

```
package controles;
```

```
import entidades.Pecas;
```

```
import interfaces.JanelaCadastroPecas;
```

```
import java.awt.Frame;
```

```
public class ControladorCadastroPecas {
```

```
    public ControladorCadastroPecas() {  
        this(null);  
    }
```

```
    public ControladorCadastroPecas(Frame owner) {  
        JanelaCadastroPecas janela = new JanelaCadastroPecas(this, owner);  
        janela.setVisible(true);  
        janelaToFront();  
        janela.requestFocus();  
    }
```

```
    public String inserirPecas(Pecas pecas) {  
        Pecas peca_buscada = Pecas.buscarPecas(pecas.getCodigo());  
        if (peca_buscada == null) return Pecas.inserirPecas(pecas);  
        else return "Código de Peça já cadastrado";  
    }
```

```
    public String alterarPecas(Pecas pecas) {  
        Pecas peca_buscada = Pecas.buscarPecas(pecas.getCodigo());  
        if (peca_buscada != null) return Pecas.alterarPecas(pecas);  
        else return "Código de Peça não cadastrado";  
    }
```

```
    public String removerPecas(int codigo) {  
        Pecas peca_buscada = Pecas.buscarPecas(codigo);  
        if (peca_buscada != null) return Pecas.removerPecas(codigo);  
        else return "Código de Peça não cadastrado";  
    }
```

}

}

\$\$\$ src.controles.ControladorCadastroPecasSinistros

```
package controles;

import entidades.Pecas;
import entidades.PecasSinistros;
import entidades.Sinistro;
import interfaces.JanelaCadastroPecasSinistros;
import java.awt.Frame;

public class ControladorCadastroPecasSinistros {

    public ControladorCadastroPecasSinistros() {
        this(null, null);
    }

    public ControladorCadastroPecasSinistros(Frame owner, Sinistro sinistro) {
        JanelaCadastroPecasSinistros janela =
            new JanelaCadastroPecasSinistros(this, sinistro, owner);
        janela.setVisible(true);
        janelaToFront();
        janela.requestFocus();
    }

    public String inserirPecasSinistros(Pecas peca, Sinistro sinistro) {
        Sinistro sinistro_buscado =
            sinistro != null ? Sinistro.buscarSinistro(sinistro.getId()) : null;
        if (sinistro_buscado == null) return "Sinistro nao cadastrado";

        Pecas peca_buscada =
            peca != null ? Pecas.buscarPecas(peca.getCodigo()) : null;
        if (peca_buscada == null) return "Peca nao cadastrada";

        if (PecasSinistros.existePecasSinistros(peca.getCodigo(), sinistro.getId())) {
            return "Peca ja associada ao sinistro";
        }
    }
}
```

```

        return PecasSinistros.inserirPecasSinistros(peca_buscada, sinistro_buscado);
    }

    public String removerPecasSinistros(Pecas peca, Sinistro sinistro) {
        Sinistro sinistro_buscado =
            sinistro != null ? Sinistro.buscarSinistro(sinistro.getId()) : null;
        if (sinistro_buscado == null) return "Sinistro nao cadastrado";

        Pecas peca_buscada =
            peca != null ? Pecas.buscarPecas(peca.getCodigo()) : null;
        if (peca_buscada == null) return "Peca nao cadastrada";

        if (!PecasSinistros.existePecasSinistros(peca.getCodigo(), sinistro.getId())) {
            return "Peca nao associada ao sinistro";
        }

        return PecasSinistros.removerPecasSinistros(peca_buscada, sinistro_buscado);
    }
}

```

\$\$\$ src.controles.ControladorCadastroSeguradoras

```
package controles;

import entidades.Seguradora;
import interfaces.JanelaCadastroSeguradoras;
import java.awt.Frame;

public class ControladorCadastroSeguradoras {

    public ControladorCadastroSeguradoras() {
        this(null);
    }

    public ControladorCadastroSeguradoras(Frame owner) {
        JanelaCadastroSeguradoras janela = new JanelaCadastroSeguradoras(this, owner);
        janela.setVisible(true);
        janelaToFront();
        janela.requestFocus();
    }

    public String inserirSeguradora(Seguradora seguradora) {
        Seguradora seguradora_buscada = Seguradora.buscarSeguradora(seguradora.getNome());
        if (seguradora_buscada == null) return Seguradora.inserirSeguradora(seguradora);
        else return "Nome de Seguradora já cadastrado";
    }

    public String alterarSeguradora(Seguradora seguradora) {
        Seguradora seguradora_buscada = Seguradora.buscarSeguradora(seguradora.getNome());
        if (seguradora_buscada != null) return Seguradora.alterarSeguradora(seguradora);
        else return "Nome de Seguradora não cadastrado";
    }

    public String removerSeguradora(String nome) {
        Seguradora seguradora_buscada = Seguradora.buscarSeguradora(nome);
        if (seguradora_buscada != null) return Seguradora.removerSeguradora(nome);
        else return "Nome de Seguradora não cadastrado";
    }
}
```

}

}

\$\$\$ src.controles.ControladorCadastroSinistros

```
package controles;

import entidades.Sinistro;
import interfaces.JanelaCadastroSinistros;
import java.awt.Frame;

public class ControladorCadastroSinistros {

    public ControladorCadastroSinistros() {
        this(null);
    }

    public ControladorCadastroSinistros(Frame owner) {
        JanelaCadastroSinistros janela = new JanelaCadastroSinistros(this, owner);
        janela.setVisible(true);
        janelaToFront();
        janela.requestFocus();
    }

    public String inserirSinistro(Sinistro sinistro) {
        return Sinistro.inserirSinistro(sinistro);
    }

    public String alterarSinistro(Sinistro sinistro) {
        Sinistro sinistro1 = Sinistro.buscarSinistro(sinistro.getId());
        if (sinistro1 != null) return Sinistro.alterarSinistro(sinistro);
        else return "Segurado de Sinistro não cadastrado";
    }

    public String removerSinistro(int id) {
        Sinistro sinistro1 = Sinistro.buscarSinistro(id);
        if (sinistro1 != null) return Sinistro.removerSinistro(id);
        else return "Sinistro não cadastrado";
    }
}
```

\$\$\$ src.entidades.Pecas

```
package entidades;

import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.Locale;
import persistência.BD;

public class Pecas {

    public enum MarcaPeca {
        OEM("oem"), ORIGINAL("original"), GENUINA("genuina");

        private final String texto;

        MarcaPeca(String texto) { this.texto = texto; }

        public static MarcaPeca fromTexto(String texto) {
            if (texto == null) throw new IllegalArgumentException("Marca da peca nao informada");
            String valor = texto.trim().toLowerCase(Locale.ROOT);
            for (MarcaPeca marca : values()) {
                if (marca.texto.equals(valor)) return marca;
            }
            throw new IllegalArgumentException("Marca da peca invalida: " + texto);
        }
    }

    @Override
    public String toString() { return texto; }
}

protected int codigo;
protected String nome;
protected MarcaPeca marca;
protected double preco;
```

```

protected boolean mao_obra_propria;
protected Integer dias_garantia;
protected String cor;

public Pecas(int codigo, String nome, MarcaPeca marca, double preco,
    boolean mao_obra_propria, Integer dias_garantia, String cor) {
    this.codigo = codigo;
    this.nome = nome;
    this.marca = marca;
    this.preco = preco;
    this.mao_obra_propria = mao_obra_propria;
    this.dias_garantia = dias_garantia;
    this.cor = cor;
}

public Pecas() { this(0, null, null, 0.0, false, null, null); }

public int getCodigo() { return codigo; }
public String getNome() { return nome; }
public MarcaPeca getMarca() { return marca; }
public MarcaPeca getCategoria() { return marca; }
public double getPreco() { return preco; }
public boolean getMaoObraPropria() { return mao_obra_propria; }
public boolean isMaoObraPropria() { return mao_obra_propria; }
public boolean getMaoDeObra() { return mao_obra_propria; }
public Integer getDiasGarantia() { return dias_garantia; }
public String getCor() { return cor; }

public void setCodigo(int codigo) { this.codigo = codigo; }
public void setNome(String nome) { this.nome = nome; }
public void setMarca(MarcaPeca marca) { this.marca = marca; }
public void setCategoria(MarcaPeca marca) { this.marca = marca; }
public void setPreco(double preco) { this.preco = preco; }
public void setMaoObraPropria(boolean valor) { this.mao_obra_propria = valor; }
public void setMaoDeObra(boolean valor) { this.mao_obra_propria = valor; }
public void setDiasGarantia(Integer valor) { this.dias_garantia = valor; }
public void setCor(String cor) { this.cor = cor; }

```

```
public Pecas getVisao() { return this; }
```

```
@Override
```

```
public String toString() { return codigo + " - " + nome + " [" + marca + "]; }
```

```
private static Pecas criarVisao(ResultSet resultado) throws SQLException {  
    int codigo = resultado.getInt("codigo");  
    String nome = resultado.getString("nome");  
    MarcaPeca marca = MarcaPeca.fromTexto(resultado.getString("marca"));  
    double preco = resultado.getDouble("preco");  
    boolean mao_obra_propria = resultado.getBoolean("mao_obra_propria");  
    Integer dias_garantia = resultado.getObject("dias_garantia") == null  
        ? null : resultado.getInt("dias_garantia");  
    String cor = resultado.getString("cor");  
  
    return new Pecas(codigo, nome, marca, preco, mao_obra_propria,  
        dias_garantia, cor);  
}
```

```
private static final String COLUNAS =  
    "codigo, nome, marca, preco, mao_obra_propria, dias_garantia, cor";
```

```
public static Pecas[] getVisoes() {  
    ArrayList<Pecas> visoes = new ArrayList<>();  
    try (PreparedStatement comando = BD.conexao.prepareStatement(  
        "SELECT " + COLUNAS + " FROM pecas");  
        ResultSet resultados = comando.executeQuery()) {  
        while (resultados.next()) visoes.add(criarVisao(resultados));  
    } catch (SQLException | IllegalArgumentException excecao) {  
        excecao.printStackTrace();  
    }  
    return visoes.toArray(new Pecas[0]);  
}
```

```
public static Pecas buscarPecas(int codigo) {  
    try (PreparedStatement comando = BD.conexao.prepareStatement(  
        "SELECT * FROM pecas WHERE codigo = ?")) {  
        ResultSet resultados = comando.executeQuery();  
        while (resultados.next()) {  
            Peca peca = criarVisao(resultados);  
            return peca;  
        }  
    }  
}
```

```

        "SELECT " + COLUNAS + " FROM pecas WHERE codigo = ?")) {
    comando.setInt(1, codigo);
    try (ResultSet resultados = comando.executeQuery()) {
        return resultados.next() ? criarVisao(resultados) : null;
    }
} catch (SQLException | IllegalArgumentException excecao) {
    excecao.printStackTrace();
    return null;
}
}

```

```

public static Pecas buscarPecas(String nome) {
    try (PreparedStatement comando = BD.conexao.prepareStatement(
        "SELECT " + COLUNAS + " FROM pecas WHERE nome = ?")) {
        comando.setString(1, nome);
        try (ResultSet resultados = comando.executeQuery()) {
            return resultados.next() ? criarVisao(resultados) : null;
        }
    } catch (SQLException | IllegalArgumentException excecao) {
        excecao.printStackTrace();
        return null;
    }
}

```

```

public static Pecas[] buscarPecasPorSinistro(int sinistroId) {
    return PecasSinistros.buscarPecasPorSinistro(sinistroId);
}

```

```

private static void preencher(PreparedStatement comando, Pecas peca) throws SQLException {
    comando.setInt(1, peca.codigo);
    comando.setString(2, peca.nome);
    comando.setString(3, peca.marca.toString());
    comando.setDouble(4, peca.preco);
    comando.setBoolean(5, peca.mao_obra_propria);
    if (peca.dias_garantia == null) comando.setNull(6, java.sql.Types.INTEGER);
    else comando.setInt(6, peca.dias_garantia);
    comando.setString(7, peca.cor);
}

```

```
}
```

```
public static String inserirPecas(Pecas peca) {  
    String sql = "INSERT INTO pecas (" + COLUNAS + ") VALUES (?, ?, ?, ?, ?, ?, ?)";  
    try (PreparedStatement comando = BD.conexao.prepareStatement(sql)) {  
        preencher(comando, peca);  
        comando.executeUpdate();  
        return null;  
    } catch (SQLException | NullPointerException excecao) {  
        excecao.printStackTrace();  
        return "Erro na insercao da peca no BD";  
    }  
}
```

```
public static String alterarPecas(Pecas peca) {  
    String sql = "UPDATE pecas SET nome=?, marca=?, preco=?, mao_obra_propria=?, "  
        + " dias_garantia=?, cor=? "  
        + " WHERE codigo=?";  
    try (PreparedStatement comando = BD.conexao.prepareStatement(sql)) {  
        comando.setString(1, peca.nome);  
        comando.setString(2, peca.marca.toString());  
        comando.setDouble(3, peca.preco);  
        comando.setBoolean(4, peca.mao_obra_propria);  
        if (peca.dias_garantia == null) comando.setNull(5, java.sql.Types.INTEGER);  
        else comando.setInt(5, peca.dias_garantia);  
        comando.setString(6, peca.cor);  
        comando.setInt(7, peca.codigo);  
        comando.executeUpdate();  
        return null;  
    } catch (SQLException | NullPointerException excecao) {  
        excecao.printStackTrace();  
        return "Erro na alteracao da peca no BD";  
    }  
}
```

```
public static String removerPecas(int codigo) {  
    try (PreparedStatement comando = BD.conexao.prepareStatement(  

```

```

        "DELETE FROM pecas WHERE codigo = ?")) {
    comando.setInt(1, codigo);
    comando.executeUpdate();
    return null;
} catch (SQLException excecao) {
    excecao.printStackTrace();
    return "Erro na remocao da peca no BD";
}
}

public static String removerPecas(String nome) {
    try (PreparedStatement comando = BD.conexao.prepareStatement(
        "DELETE FROM pecas WHERE nome = ?")) {
        comando.setString(1, nome);
        comando.executeUpdate();
        return null;
    } catch (SQLException excecao) {
        excecao.printStackTrace();
        return "Erro na remocao da peca no BD";
    }
}
}

```

\$\$\$ src.entidades.Seguradora

```
package entidades;

import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import persistência.BD;

public class PecasSinistros {

    private final int peca_codigo;
    private final int sinistro_id;

    public PecasSinistros(int peca_codigo, int sinistro_id) {
        this.pecas_codigo = peca_codigo;
        this.sinistro_id = sinistro_id;
    }

    public PecasSinistros(Pecas peca, Sinistro sinistro) {
        this(pecas != null ? peca.getCodigo() : 0,
            sinistro != null ? sinistro.getId() : 0);
    }

    public int getPecaCodigo() {
        return peca_codigo;
    }

    public int getSinistroId() {
        return sinistro_id;
    }

    public String toString() {
        return peca_codigo + " / " + sinistro_id;
    }
}
```



```

private static Pecas criarPeca(ResultSet resultado) throws SQLException {
    int codigo = resultado.getInt("codigo");
    String nome = resultado.getString("nome");
    Pecas.MarcaPeca marca = Pecas.MarcaPeca.fromTexto(resultado.getString("marca"));
    double preco = resultado.getDouble("preco");
    boolean mao_obra_propria = resultado.getBoolean("mao_obra_propria");
    Integer dias_garantia = resultado.getObject("dias_garantia") == null
        ? null : resultado.getInt("dias_garantia");
    String cor = resultado.getString("cor");

    return new Pecas(codigo, nome, marca, preco, mao_obra_propria,
        dias_garantia, cor);
}

```

```

public static Pecas[] buscarPecasPorSinistro(int sinistroId) {
    ArrayList<Pecas> visoes = new ArrayList<>();
    String sql = "SELECT p.codigo, p.nome, p.marca, p.preco, p.mao_obra_propria, "
        + "p.dias_garantia, p.cor "
        + "FROM pecas_sinistros ps "
        + "JOIN pecas p ON p.codigo = ps.pecas_codigo "
        + "WHERE ps.sinistro_id = ?";

    try (PreparedStatement comando = BD.conexao.prepareStatement(sql)) {
        comando.setInt(1, sinistroId);
        try (ResultSet resultados = comando.executeQuery()) {
            while (resultados.next()) {
                try {
                    visoes.add(criarPeca(resultados));
                } catch (IllegalArgumentException excecao_enum) {
                    // Ignora peças com dados fora do domínio esperado.
                }
            }
        }
    } catch (SQLException excecao_sql) {
        excecao_sql.printStackTrace();
    }
}

```

```
return visoes.toArray(new Pecas[0]);  
}
```

```
public static boolean existePecasSinistros(int peca_codigo, int sinistro_id) {  
    String sql = "SELECT COUNT(*) FROM pecas_sinistros WHERE peca_codigo = ? AND  
sinistro_id = ?";
```

```
    try (PreparedStatement comando = BD.conexao.prepareStatement(sql)) {  
        comando.setInt(1, peca_codigo);  
        comando.setInt(2, sinistro_id);  
        try (ResultSet resultados = comando.executeQuery()) {  
            return resultados.next() && resultados.getInt(1) > 0;  
        }  
    } catch (SQLException excecao_sql) {  
        excecao_sql.printStackTrace();  
        return false;  
    }  
}
```

```
public static String inserirPecasSinistros(Pecas peca, Sinistro sinistro) {  
    if (peca == null || sinistro == null) {  
        return "Peca ou sinistro nao informado";  
    }
```

```
    String sql = "INSERT INTO pecas_sinistros (peca_codigo, sinistro_id) VALUES (?, ?)";
```

```
    try (PreparedStatement comando = BD.conexao.prepareStatement(sql)) {  
        comando.setInt(1, peca.getCodigo());  
        comando.setInt(2, sinistro.getId());  
        comando.executeUpdate();  
        return null;  
    } catch (SQLException excecao_sql) {  
        excecao_sql.printStackTrace();  
        return "Erro na insercao da associacao peca/sinistro no BD";  
    }  
}
```

```
public static String removerPecasSinistros(Pecas peca, Sinistro sinistro) {  
    if (peca == null || sinistro == null) {  
        return "Peca ou sinistro nao informado";  
    }  
}
```

```
String sql = "DELETE FROM pecas_sinistros WHERE peca_codigo = ? AND sinistro_id  
= ?";
```

```
try (PreparedStatement comando = BD.conexao.prepareStatement(sql)) {  
    comando.setInt(1, peca.getCodigo());  
    comando.setInt(2, sinistro.getId());  
    comando.executeUpdate();  
    return null;  
} catch (SQLException excecao_sql) {  
    excecao_sql.printStackTrace();  
    return "Erro na remocao da associacao peca/sinistro no BD";  
}  
}  
}
```

\$\$\$ src.entidades.Seguradora

```
package entidades;

import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.Locale;
import persistência.BD;

public class Seguradora {

    public enum FormaPagamentoPreferencial {
        BOLETO("boleto"),
        CARTAO("cartao"),
        PIX("pix"),
        DEBITO_AUTOMATICO("debito_automatico");

        private final String texto;

        FormaPagamentoPreferencial(String texto) {
            this.texto = texto;
        }

        public static FormaPagamentoPreferencial fromTexto(String texto) {
            if (texto == null) {
                throw new IllegalArgumentException("Forma de pagamento não informada");
            }

            String normalizado = texto.trim().toLowerCase(Locale.ROOT);

            if (normalizado.equals("boleto")) return BOLETO;
            if (normalizado.equals("cartao") || normalizado.equals("cartão")) return CARTAO;
            if (normalizado.equals("pix")) return PIX;
            if (normalizado.equals("debito_automatico")
                || normalizado.equals("debito automatico"))
```

```
        || normalized.equals("débito_automático")
        || normalized.equals("débito automático")) {
    return DEBITO_AUTOMATICO;
}
```

```
    throw new IllegalArgumentException("Forma de pagamento inválida: " + texto);
}
```

```
@Override
public String toString() {
    return texto;
}
}
```

```
private String nome;
private String cidade;
private double cobertura_percentual;
private boolean possui_atendimento_24h;
private FormaPagamentoPreferencial forma_pagamento_preferencial;
```

```
public String getNome() { return nome; }
public String getCidade() { return cidade; }
public double getCoberturaPercentual() { return cobertura_percentual; }
public boolean getPossuiAtendimento24h() { return possui_atendimento_24h; }
public boolean isPossuiAtendimento24h() { return possui_atendimento_24h; }
public FormaPagamentoPreferencial getFormaPagamentoPreferencial() { return
forma_pagamento_preferencial; }
```

```
public void setCidade(String cidade) { this.cidade = cidade; }
public void setCoberturaPercentual(double cobertura_percentual) { this.cobertura_percentual =
cobertura_percentual; }
public void setPossuiAtendimento24h(boolean possui_atendimento_24h)
{ this.possui_atendimento_24h = possui_atendimento_24h; }
public void setFormaPagamentoPreferencial(FormaPagamentoPreferencial
forma_pagamento_preferencial) {
    this.forma_pagamento_preferencial = forma_pagamento_preferencial;
}
```

```

public Seguradora(
    String nome,
    String cidade,
    double cobertura_percentual,
    boolean possui_atendimento_24h,
    FormaPagamentoPreferencial forma_pagamento_preferencial
) {
    this.nome = nome;
    this.cidade = cidade;
    this.cobertura_percentual = cobertura_percentual;
    this.possui_atendimento_24h = possui_atendimento_24h;
    this.forma_pagamento_preferencial = forma_pagamento_preferencial;
}

public Seguradora(String nome, String cidade, double cobertura_percentual) {
    this(nome, cidade, cobertura_percentual, false, FormaPagamentoPreferencial.BOLETO);
}

public Seguradora(String nome, String cidade) {
    this(nome, cidade, 0, false, FormaPagamentoPreferencial.BOLETO);
}

public String toString() {
    return nome + " [" + cidade + "]";
}

public Seguradora getVisao() {
    return new Seguradora(
        nome,
        cidade,
        cobertura_percentual,
        possui_atendimento_24h,
        forma_pagamento_preferencial
    );
}

public static Seguradora[] getVisoes() {

```

```

String sql = "SELECT nome, cidade, cobertura_percentual, possui_atendimento_24h,
forma_pagamento_preferencial FROM seguradoras";

ResultSet lista_resultados = null;
ArrayList<Seguradora> visoes = new ArrayList<>();

try {
    PreparedStatement comando = BD.conexao.prepareStatement(sql);
    lista_resultados = comando.executeQuery();

    while (lista_resultados.next()) {
        String nome = lista_resultados.getString("nome");
        String cidade = lista_resultados.getString("cidade");
        double cobertura_percentual = lista_resultados.getDouble("cobertura_percentual");
        boolean possui_atendimento_24h =
lista_resultados.getBoolean("possui_atendimento_24h");
        FormaPagamentoPreferencial forma_pagamento_preferencial =
            FormaPagamentoPreferencial.fromTexto(
                lista_resultados.getString("forma_pagamento_preferencial")
            );
        visoes.add(new Seguradora(
            nome,
            cidade,
            cobertura_percentual,
            possui_atendimento_24h,
            forma_pagamento_preferencial
        ));
    }

    lista_resultados.close();
    comando.close();

} catch (SQLException excecao_sql) {
    excecao_sql.printStackTrace();
}

return visoes.toArray(new Seguradora[visoes.size()]);
}

```

```

public static Seguradora buscarSeguradora(String nome) {
    String sql = "SELECT * FROM seguradoras WHERE nome = ?";
    ResultSet lista_resultados = null;
    Seguradora seguradora = null;

    try {
        PreparedStatement comando = BD.conexao.prepareStatement(sql);
        comando.setString(1, nome);
        lista_resultados = comando.executeQuery();

        while (lista_resultados.next()) {
            seguradora = new Seguradora(
                nome,
                lista_resultados.getString("cidade"),
                lista_resultados.getDouble("cobertura_percentual"),
                lista_resultados.getBoolean("possui_atendimento_24h"),
                FormaPagamentoPreferencial.fromTexto(
                    lista_resultados.getString("forma_pagamento_preferencial")
                )
            );
        }

        lista_resultados.close();
        comando.close();

    } catch (SQLException excecao_sql) {
        excecao_sql.printStackTrace();
        seguradora = null;
    }

    return seguradora;
}

```

```

public static String inserirSeguradora(Seguradora seguradora) {
    String sql = "INSERT INTO seguradoras (nome, cidade, cobertura_percentual,
    possui_atendimento_24h, forma_pagamento_preferencial) VALUES (?, ?, ?, ?, ?)";
}

```



```

try {
    PreparedStatement comando = BD.conexao.prepareStatement(sql);

    comando.setString(1, seguradora.getNome());
    comando.setString(2, seguradora.getCidade());
    comando.setDouble(3, seguradora.getCoberturaPercentual());
    comando.setBoolean(4, seguradora.getPossuiAtendimento24h());
    comando.setString(5, seguradora.getFormaPagamentoPreferencial().toString());

    comando.executeUpdate();
    comando.close();

    return null;

} catch (SQLException excecao_sql) {
    excecao_sql.printStackTrace();
    return "Erro na Inserção da Seguradora no BD";
}
}

```

```

public static String alterarSeguradora(Seguradora seguradora) {
    String sql = "UPDATE seguradoras SET cidade = ?, cobertura_percentual = ?,
possui_atendimento_24h = ?, forma_pagamento_preferencial = ?"
        + " WHERE nome = ?";

```

```

try {
    PreparedStatement comando = BD.conexao.prepareStatement(sql);

    comando.setString(1, seguradora.getCidade());
    comando.setDouble(2, seguradora.getCoberturaPercentual());
    comando.setBoolean(3, seguradora.getPossuiAtendimento24h());
    comando.setString(4, seguradora.getFormaPagamentoPreferencial().toString());
    comando.setString(5, seguradora.getNome());

    comando.executeUpdate();
    comando.close();

```

```

        return null;

    } catch (SQLException excecao_sql) {
        excecao_sql.printStackTrace();
        return "Erro na Alteração da Seguradora no BD";
    }
}

public static String removerSeguradora(String nome) {
    String sql = "DELETE FROM seguradoras WHERE nome = ?";

    try {
        PreparedStatement comando = BD.conexao.prepareStatement(sql);

        comando.setString(1, nome);
        comando.executeUpdate();
        comando.close();

        return null;

    } catch (SQLException excecao_sql) {
        excecao_sql.printStackTrace();
        return "Erro na Remoção da Seguradora no BD";
    }
}
}

```

\$\$\$ src.entidades.Sinistro

```
package entidades;

import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.ArrayList;
import java.util.Locale;
import persistência.BD;

public class Sinistro {

    public enum GrauMonta {
        PEQUENA("pequena"), MEDIA("media"), GRANDE("grande");
        private final String texto;
        GrauMonta(String texto) { this.texto = texto; }
        public static GrauMonta fromTexto(String texto) {
            if (texto == null) throw new IllegalArgumentException("Grau de monta não informado");
            String normalizado = texto.trim().toLowerCase(Locale.ROOT)
                .replace("á", "a").replace("à", "a").replace("â", "a")
                .replace("ã", "a").replace("é", "e").replace("ê", "e")
                .replace("í", "i").replace("ó", "o").replace("ô", "o")
                .replace("õ", "o").replace("ú", "u").replace("ç", "c");
            if (normalizado.contains("pequena")) return PEQUENA;
            if (normalizado.contains("media")) return MEDIA;
            if (normalizado.contains("grande")) return GRANDE;
            throw new IllegalArgumentException("Grau de monta inválido: " + texto);
        }
        @Override public String toString() { return texto; }
    }

    private int id;
    private String segurado;
    private String telefone;
    private String cidade;
```

```

private GrauMonta grau_monta;
private boolean perda_total;
private ArrayList<Pecas> pecas = new ArrayList<Pecas>();

public Sinistro(int id, String segurado, String telefone, String cidade,
    GrauMonta grau_monta, boolean perda_total) {
    this.id = id;
    this.segurado = segurado;
    this.telefone = telefone;
    this.cidade = cidade;
    this.grau_monta = grau_monta;
    this.perda_total = perda_total;
}

public Sinistro(String segurado, String telefone, String cidade,
    GrauMonta grau_monta, boolean perda_total) {
    this(0, segurado, telefone, cidade, grau_monta, perda_total);
}

public int getId() { return id; }
public String getSegurado() { return segurado; }
public String getTelefone() { return telefone; }
public String getCidade() { return cidade; }
public GrauMonta getGrauMonta() { return grau_monta; }
public boolean getPerdaTotal() { return perda_total; }
public boolean isPerdaTotal() { return perda_total; }
public Pecas[] getPecas() { return pecas.toArray(new Pecas[0]); }
public void setSegurado(String segurado) { this.segurado = segurado; }
public void setTelefone(String telefone) { this.telefone = telefone; }
public void setCidade(String cidade) { this.cidade = cidade; }
public void setGrauMonta(GrauMonta grau_monta) { this.grau_monta = grau_monta; }
public void setPerdaTotal(boolean perda_total) { this.perda_total = perda_total; }
public void setPecas(Pecas[] pecas) {
    this.pecas = new ArrayList<Pecas>();
    if (pecas != null) for (Pecas peca : pecas) if (peca != null) this.pecas.add(peca);
}

```

@Override

```
public String toString() {  
    return "#" + id + " - " + segurado + " - " + (cidade != null ? cidade : "")  
        + " (" + grau_monta + ")";  
}
```

```
private static Sinistro criarVisao(ResultSet resultado) throws SQLException {  
    return new Sinistro(resultado.getInt("id"), resultado.getString("segurado"),  
        resultado.getString("telefone"), resultado.getString("cidade"),  
        GrauMonta.fromTexto(resultado.getString("grau_monta")),  
        resultado.getBoolean("perda_total"));  
}
```

```
public static Sinistro[] getVisoes() {  
    ArrayList<Sinistro> visoes = new ArrayList<Sinistro>();  
    String sql = "SELECT id, segurado, telefone, cidade, grau_monta, perda_total FROM sinistros  
ORDER BY id";  
    try (PreparedStatement comando = BD.conexao.prepareStatement(sql);  
        ResultSet resultados = comando.executeQuery()) {  
        while (resultados.next()) visoes.add(criarVisao(resultados));  
    } catch (SQLException | IllegalArgumentException excecao) { excecao.printStackTrace(); }  
    return visoes.toArray(new Sinistro[0]);  
}
```

```
public static Sinistro buscarSinistro(int id) {  
    String sql = "SELECT id, segurado, telefone, cidade, grau_monta, perda_total FROM sinistros  
WHERE id = ?";  
    try (PreparedStatement comando = BD.conexao.prepareStatement(sql)) {  
        comando.setInt(1, id);  
        try (ResultSet resultados = comando.executeQuery()) {  
            if (!resultados.next()) return null;  
            Sinistro sinistro = criarVisao(resultados);  
            sinistro.setPecas(PecasSinistros.buscarPecasPorSinistro(id));  
            return sinistro;  
        }  
    } catch (SQLException | IllegalArgumentException excecao) {  
        excecao.printStackTrace();  
    }
```

```
        return null;
    }
}
```

```
public static String inserirSinistro(Sinistro sinistro) {
    String sql = "INSERT INTO sinistros (segurado, telefone, cidade, grau_monta, perda_total)
VALUES (?, ?, ?, ?, ?)";

    try (PreparedStatement comando = BD.conexao.prepareStatement(sql,
Statement.RETURN_GENERATED_KEYS)) {
        comando.setString(1, sinistro.getSegurado());
        comando.setString(2, sinistro.getTelefone());
        comando.setString(3, sinistro.getCidade());
        comando.setString(4, sinistro.getGrauMonta().toString());
        comando.setBoolean(5, sinistro.getPerdaTotal());
        comando.executeUpdate();
        try (ResultSet chaves = comando.getGeneratedKeys()) {
            if (chaves.next()) sinistro.id = chaves.getInt(1);
        }
        return sinistro.id > 0 ? null : "Erro ao obter o ID do Sinistro criado";
    } catch (SQLException excecao) {
        excecao.printStackTrace();
        return "Erro na Insercao do Sinistro no BD";
    }
}
```

```
public static String alterarSinistro(Sinistro sinistro) {
    String sql = "UPDATE sinistros SET asegurado = ?, telefone = ?, cidade = ?, grau_monta = ?,
perda_total = ? WHERE id = ?";

    try (PreparedStatement comando = BD.conexao.prepareStatement(sql)) {
        comando.setString(1, sinistro.getSegurado());
        comando.setString(2, sinistro.getTelefone());
        comando.setString(3, sinistro.getCidade());
        comando.setString(4, sinistro.getGrauMonta().toString());
        comando.setBoolean(5, sinistro.getPerdaTotal());
        comando.setInt(6, sinistro.getId());
        return comando.executeUpdate() == 1 ? null : "Sinistro não cadastrado";
    } catch (SQLException excecao) {
        excecao.printStackTrace();
    }
}
```

```

        return "Erro na Alteracao do Sinistro no BD";
    }
}

public static String removerSinistro(int id) {
    try (PreparedStatement comando = BD.conexao.prepareStatement("DELETE FROM sinistros
WHERE id = ?")) {
        comando.setInt(1, id);
        return comando.executeUpdate() == 1 ? null : "Sinistro não cadastrado";
    } catch (SQLException excecao) {
        excecao.printStackTrace();
        return "Erro na Remocao do Sinistro no BD";
    }
}
}

```

\$\$\$ src.interfaces.JanelaCadastroPecas

```
package interfaces;

import javax.swing.JOptionPane;
import javax.swing.ButtonGroup;
import controles.ControladorCadastroPecas;
import entidades.Pecas;
import javax.swing.DefaultComboBoxModel;
import java.awt.Frame;

public class JanelaCadastroPecas extends javax.swing.JFrame {

    ControladorCadastroPecas controlador;
    Pecas[] pecas_cadastradas;

    public JanelaCadastroPecas(ControladorCadastroPecas controlador) {
        this(controlador, null);
    }

    public JanelaCadastroPecas(
        ControladorCadastroPecas controlador,
        Frame owner
    ) {
        this.controlador = controlador;
        pecas_cadastradas = Pecas.getVisoes();
        initComponents();
        atualizarPecasCadastradas();
        setSize(new java.awt.Dimension(720, 560));
        configurarJanelaDependente(owner);
        limparCampos(null);
    }

    private void configurarJanelaDependente(Frame owner) {
        setAutoRequestFocus(true);
        setAlwaysOnTop(true);
    }
}
```



```

    if (owner == null) {
        setLocationByPlatform(true);
        return;
    }

    posicionarRelativaAoOwner(owner);
}

private void posicionarRelativaAoOwner(Frame owner) {
    int x = owner.getX() + (owner.getWidth() - getWidth()) / 2;
    int y = owner.getY() + (owner.getHeight() - getHeight()) / 2;
    setLocation(x, y);
}

private Pecas localizarPeca(int codigo) {
    for (Pecas visao : pecas_cadastradas) {
        if (visao.getCodigo() == codigo) return visao;
    }
    return null;
}

private void atualizarPecasCadastradas() {
    atualizarPecasCadastradas(-1);
}

private void atualizarPecasCadastradas(int codigo_selecionado) {
    pecas_cadastradas = Pecas.getVisoes();
    pecas_cadastradasComboBox.setModel(
        new DefaultComboBoxModel(pecas_cadastradas)
    );

    Pecas visao_selecionada = localizarPeca(codigo_selecionado);
    if (visao_selecionada != null) {
        pecas_cadastradasComboBox.setSelectedItem(visao_selecionada);
    }
}

```

```
private void informarErro(String mensagem) {
    JOptionPane.showMessageDialog(
        this,
        mensagem,
        "Erro",
        JOptionPane.ERROR_MESSAGE
    );
}
```

```
private Boolean obterMaoDeObraInformada() {
    if (maoDeObraSimRadioButton.isSelected()) return true;
    if (maoDeObraNaoRadioButton.isSelected()) return false;
    return null;
}
```

```
private void definirMaoDeObra(Boolean valor) {
    maoDeObraButtonGroup.clearSelection();
    if (valor == null) return;

    if (valor) {
        maoDeObraSimRadioButton.setSelected(true);
    } else {
        maoDeObraNaoRadioButton.setSelected(true);
    }
}
```

```
private Pecas obterPecasInformada() {
    String codigoStr = codigoTextField.getText();
    if (codigoStr.isEmpty()) {
        throw new IllegalArgumentException("Informe o código da peça.");
    }

    int codigo;
    try {
        codigo = Integer.parseInt(codigoStr);
    } catch (NumberFormatException e) {
        throw new IllegalArgumentException("Código da peça inválido.");
    }
}
```

```
}
```

```
String nome = nomeTextField.getText();
```

```
if (nome.isEmpty()) {
```

```
    throw new IllegalArgumentException("Informe o nome da peça.");
```

```
}
```

```
Pecas.MarcaPeca marca =
```

```
    (Pecas.MarcaPeca) marcaComboBox.getSelectedItem();
```

```
if (marca == null) {
```

```
    throw new IllegalArgumentException("Informe a marca da peça.");
```

```
}
```

```
String precoStr = precoTextField.getText();
```

```
if (precoStr.isEmpty()) {
```

```
    throw new IllegalArgumentException("Informe o preço da peça.");
```

```
}
```

```
double preco;
```

```
try {
```

```
    preco = Double.parseDouble(precoStr.replace(",", "."));
```

```
} catch (NumberFormatException e) {
```

```
    throw new IllegalArgumentException("Preço da peça inválido.");
```

```
}
```

```
String cor = corTextField.getText();
```

```
String prazoGarantiaStr = prazoGarantiaTextField.getText();
```

```
Integer prazoGarantia = null;
```

```
if (!prazoGarantiaStr.isEmpty()) {
```

```
    try {
```

```
        prazoGarantia = Integer.parseInt(prazoGarantiaStr);
```

```
    } catch (NumberFormatException e) {
```

```
        throw new IllegalArgumentException("Prazo de garantia inválido.");
```

```
    }
```

```
}
```

```
Boolean mao_de_obra = obterMaoDeObraInformada();
```

```

        if (mao_de_obra == null) {
            throw new IllegalArgumentException("Selecione se a peça possui mão de obra.");
        }

        return new Pecas(
            codigo, nome, marca, preco, mao_de_obra, prazoGarantia,
            cor.isEmpty() ? null : cor
        );
    }

```

```

private Pecas getVisaoAlterada(int codigo) {
    for (Pecas visao : pecas_cadastradas) {
        if (visao.getCodigo() == codigo) return visao;
    }
    return null;
}

```

```

@SuppressWarnings("unchecked")
// <editor-fold defaultstate="collapsed" desc="Generated Code">
private void initComponents() {
    java.awt.GridBagConstraints gridBagConstraints;

    comandosPanel = new javax.swing.JPanel();
    inserirPecas = new javax.swing.JButton();
    alterarPecas = new javax.swing.JButton();
    consultarPecas = new javax.swing.JButton();
    removerPecas = new javax.swing.JButton();
    limparCampos = new javax.swing.JButton();
    codigoLabel = new javax.swing.JLabel();
    codigoTextField = new javax.swing.JTextField();
    nomeLabel = new javax.swing.JLabel();
    nomeTextField = new javax.swing.JTextField();
    categoriaLabel = new javax.swing.JLabel();
    categoriaTextField = new javax.swing.JTextField();
    precoLabel = new javax.swing.JLabel();
    precoTextField = new javax.swing.JTextField();
    tipoLabel = new javax.swing.JLabel();

```

```

tipoTextField = new javax.swing.JTextField();
corLabel = new javax.swing.JLabel();
corTextField = new javax.swing.JTextField();
maoDeObraLabel = new javax.swing.JLabel();
maoDeObraPanel = new javax.swing.JPanel();
maoDeObraSimRadioButton = new javax.swing.JRadioButton();
maoDeObraNaoRadioButton = new javax.swing.JRadioButton();
pecas_cadastradasComboBox = new javax.swing.JComboBox();
pecasCadastradasLabel = new javax.swing.JLabel();

setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);
setTitle("Cadastrar Peças");
setMinimumSize(new java.awt.Dimension(640, 420));
setPreferredSize(new java.awt.Dimension(720, 560));
java.awt.GridBagLayout layout = new java.awt.GridBagLayout();
layout.rowHeights = new int[] {8};
getContentPane().setLayout(layout);

inserirPecas.setText("Inserir");
inserirPecas.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        inserirPecas(evt);
    }
});
comandosPanel.add(inserirPecas);

alterarPecas.setText("Alterar");
alterarPecas.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        alterarPecas(evt);
    }
});
comandosPanel.add(alterarPecas);

consultarPecas.setText("Consultar");
consultarPecas.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {

```

```
        consultarPecas(evt);
    }
});
comandosPanel.add(consultarPecas);
```

```
removerPecas.setText("Remover");
removerPecas.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        removerPecas(evt);
    }
});
comandosPanel.add(removerPecas);
```

```
limparCampos.setText("Limpar");
limparCampos.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        limparCampos(evt);
    }
});
comandosPanel.add(limparCampos);
```

```
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 9;
gridBagConstraints.insets = new java.awt.Insets(5, 5, 5, 5);
getContentPane().add(comandosPanel, gridBagConstraints);
```

```
codigoLabel.setText("Código");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 1;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(codigoLabel, gridBagConstraints);
```

```
codigoTextField.setColumns(50);
codigoTextField.setPreferredSize(new java.awt.Dimension(80, 20));
```

```
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 1;
gridBagConstraints.ipadx = 399;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(codigoTextField, gridBagConstraints);
```

```
nomeLabel.setText("Nome");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 2;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(nomeLabel, gridBagConstraints);
```

```
nomeTextField.setColumns(50);
nomeTextField.setPreferredSize(new java.awt.Dimension(240, 20));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 2;
gridBagConstraints.ipadx = 200;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(nomeTextField, gridBagConstraints);
```

```
categoriaLabel.setText("Categoria");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 3;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(categoriaLabel, gridBagConstraints);
```

```
categoriaTextField.setColumns(50);
categoriaTextField.setPreferredSize(new java.awt.Dimension(180, 20));
gridBagConstraints = new java.awt.GridBagConstraints();
```

```
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 3;
gridBagConstraints.ipadx = 100;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(categoriaTextField, gridBagConstraints);
```

```
precoLabel.setText("Preço");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 4;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(precoLabel, gridBagConstraints);
```

```
precoTextField.setColumns(50);
precoTextField.setPreferredSize(new java.awt.Dimension(100, 20));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 4;
gridBagConstraints.ipadx = 18;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(precoTextField, gridBagConstraints);
```

```
tipoLabel.setText("Tipo");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 5;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(tipoLabel, gridBagConstraints);
```

```
tipoTextField.setColumns(14);
tipoTextField.setPreferredSize(new java.awt.Dimension(140, 20));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
```



```
gridBagConstraints.gridy = 5;  
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;  
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);  
getContentPane().add(tipoTextField, gridBagConstraints);
```

```
corLabel.setText("Cor");  
gridBagConstraints = new java.awt.GridBagConstraints();  
gridBagConstraints.gridx = 0;  
gridBagConstraints.gridy = 6;  
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;  
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);  
getContentPane().add(corLabel, gridBagConstraints);
```

```
corTextField.setColumns(10);  
corTextField.setPreferredSize(new java.awt.Dimension(100, 20));  
gridBagConstraints = new java.awt.GridBagConstraints();  
gridBagConstraints.gridx = 1;  
gridBagConstraints.gridy = 6;  
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;  
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);  
getContentPane().add(corTextField, gridBagConstraints);
```

```
maoDeObraLabel.setText("Mão de Obra");  
gridBagConstraints = new java.awt.GridBagConstraints();  
gridBagConstraints.gridx = 0;  
gridBagConstraints.gridy = 7;  
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;  
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);  
getContentPane().add(maoDeObraLabel, gridBagConstraints);
```

```
maoDeObraPanel.setLayout(new java.awt.FlowLayout(java.awt.FlowLayout.LEFT));
```

```
maoDeObraSimRadioButton.setText("Sim");  
maoDeObraPanel.add(maoDeObraSimRadioButton);
```

```
maoDeObraNaoRadioButton.setText("Não");  
maoDeObraPanel.add(maoDeObraNaoRadioButton);
```

```
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 7;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(maoDeObraPanel, gridBagConstraints);
```

```
pecas_cadastradasComboBox.setModel(new DefaultComboBoxModel(pecas_cadastradas));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 0;
gridBagConstraints.anchor = java.awt.GridBagConstraints.NORTHWEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(pecas_cadastradasComboBox, gridBagConstraints);
```

```
pecasCadastradasLabel.setText("Peças Cadastradas");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 0;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(pecasCadastradasLabel, gridBagConstraints);
```

```
pack();
} // </editor-fold>
```

```
private void inserirPecas(java.awt.event.ActionEvent evt) {
    Pecas pecas;
    String mensagem_erro = null;

    try {
        pecas = obterPecasInformada();
    } catch (IllegalArgumentException excecao) {
        informarErro(excecao.getMessage());
        return;
    }
}
```

```

mensagem_erro = controlador.inserirPecas(pecas);

if (mensagem_erro == null) {
    Pecas visao = pecas.getVisao();
    atualizarPecasCadastradas(visao.getCodigo());
} else {
    informarErro(mensagem_erro);
}
}

private void alterarPecas(java.awt.event.ActionEvent evt) {
    Pecas pecas;
    String mensagem_erro = null;

    try {
        pecas = obterPecasInformada();
    } catch (IllegalArgumentException excecao) {
        informarErro(excecao.getMessage());
        return;
    }

    mensagem_erro = controlador.alterarPecas(pecas);

    if (mensagem_erro == null) {
        Pecas visao = getVisaoAlterada(pecas.getCodigo());

        if (visao != null) {
            visao.setCodigo(pecas.getCodigo());
            visao.setNome(pecas.getNome());
            visao.setMarca(pecas.getMarca());
            visao.setPreco(pecas.getPreco());
            visao.setCor(pecas.getCor());
            visao.setMaoDeObra(pecas.getMaoDeObra());
            visao.setDiasGarantia(pecas.getDiasGarantia());

            pecas_cadastradasComboBox.updateUI();
        }
    }
}

```

```

    }
} else {
    informarErro(mensagem_erro);
}
}

```

```

private void consultarPecas(java.awt.event.ActionEvent evt) {
    Pecas visao =
        (Pecas) pecas_cadastradasComboBox.getSelectedItem();

    Pecas pecas = null;
    String mensagem_erro = null;

    if (visao != null) {
        pecas = Pecas.buscarPecas(visao.getCodigo());

        if (pecas == null)
            mensagem_erro = "Peça não cadastrada";
        } else {
            mensagem_erro = "Nenhuma Peça selecionada";
        }

    if (mensagem_erro == null) {

        codigoTextField.setText(
            String.valueOf(pecas.getCodigo())
        );

        nomeTextField.setText(pecas.getNome());
        marcaComboBox.setSelectedItem(pecas.getMarca());

        precoTextField.setText(
            String.valueOf(pecas.getPreco())
        );

        corTextField.setText(pecas.getCor() != null ? pecas.getCor() : "");
        prazoGarantiaTextField.setText(

```

```

        pecas.getDiasGarantia() != null
            ? String.valueOf(pecas.getDiasGarantia())
            : ""
    );

    definirMaoDeObra(pecas.getMaoDeObra());

} else {
    informarErro(mensagem_erro);
}
}

private void removerPecas(java.awt.event.ActionEvent evt) {
    Pecas visao =
        (Pecas) pecas_cadastradasComboBox.getSelectedItem();

    String mensagem_erro = null;

    if (visao != null)
        mensagem_erro = controlador.removerPecas(visao.getCodigo());
    else
        mensagem_erro = "Nenhuma Peça selecionada";

    if (mensagem_erro == null) {
        atualizarPecasCadastradas();
        limparCampos(evt);
    } else {
        informarErro(mensagem_erro);
    }
}

private void limparCampos(java.awt.event.ActionEvent evt) {
    codigoTextField.setText("");
    nomeTextField.setText("");
    marcaComboBox.setSelectedIndex(-1);
    precoTextField.setText("");
    corTextField.setText("");
}

```

```
    prazoGarantiaTextField.setText("");
    maoDeObraButtonGroup.clearSelection();
}
```

```
// Variables declaration - do not modify
```

```
private javax.swing.JButton alterarPecas;
private javax.swing.JLabel categoriaLabel;
private javax.swing.JTextField categoriaTextField;
private javax.swing.JLabel codigoLabel;
private javax.swing.JTextField codigoTextField;
private javax.swing.JPanel comandosPanel;
private javax.swing.JButton consultarPecas;
private javax.swing.JLabel corLabel;
private javax.swing.JTextField corTextField;
private javax.swing.JButton inserirPecas;
private javax.swing.JButton limparCampos;
private javax.swing.JLabel maoDeObraLabel;
private javax.swing.JRadioButton maoDeObraNaoRadioButton;
private javax.swing.JPanel maoDeObraPanel;
private javax.swing.JRadioButton maoDeObraSimRadioButton;
private javax.swing.JLabel nomeLabel;
private javax.swing.JTextField nomeTextField;
private javax.swing.JLabel pecasCadastradasLabel;
private javax.swing.JComboBox pecas_cadastradasComboBox;
private javax.swing.JLabel precoLabel;
private javax.swing.JTextField precoTextField;
private javax.swing.JButton removerPecas;
private javax.swing.JLabel tipoLabel;
private javax.swing.JTextField tipoTextField;
// End of variables declaration
```

```
}
```

\$\$\$ src.interfaces.JanelaCadastroPecasSinistros

```
package interfaces;

import javax.swing.JOptionPane;
import javax.swing.ButtonGroup;
import controles.ControladorCadastroPecas;
import entidades.Pecas;
import javax.swing.DefaultComboBoxModel;
import java.awt.Frame;

public class JanelaCadastroPecas extends javax.swing.JFrame {

    ControladorCadastroPecas controlador;
    Pecas[] pecas_cadastradas;

    public JanelaCadastroPecas(ControladorCadastroPecas controlador) {
        this(controlador, null);
    }

    public JanelaCadastroPecas(
        ControladorCadastroPecas controlador,
        Frame owner
    ) {
        this.controlador = controlador;
        pecas_cadastradas = Pecas.getVisoes();
        initComponents();
        atualizarPecasCadastradas();
        setSize(new java.awt.Dimension(720, 560));
        configurarJanelaDependente(owner);
        limparCampos(null);
    }

    private void configurarJanelaDependente(Frame owner) {
        setAutoRequestFocus(true);
        setAlwaysOnTop(true);
    }
}
```

```

    if (owner == null) {
        setLocationByPlatform(true);
        return;
    }

    posicionarRelativaAoOwner(owner);
}

private void posicionarRelativaAoOwner(Frame owner) {
    int x = owner.getX() + (owner.getWidth() - getWidth()) / 2;
    int y = owner.getY() + (owner.getHeight() - getHeight()) / 2;
    setLocation(x, y);
}

private Pecas localizarPeca(int codigo) {
    for (Pecas visao : pecas_cadastradas) {
        if (visao.getCodigo() == codigo) return visao;
    }
    return null;
}

private void atualizarPecasCadastradas() {
    atualizarPecasCadastradas(-1);
}

private void atualizarPecasCadastradas(int codigo_selecionado) {
    pecas_cadastradas = Pecas.getVisoes();
    pecas_cadastradasComboBox.setModel(
        new DefaultComboBoxModel(pecas_cadastradas)
    );

    Pecas visao_selecionada = localizarPeca(codigo_selecionado);
    if (visao_selecionada != null) {
        pecas_cadastradasComboBox.setSelectedItem(visao_selecionada);
    }
}

```



```
private void informarErro(String mensagem) {  
    JOptionPane.showMessageDialog(  
        this,  
        mensagem,  
        "Erro",  
        JOptionPane.ERROR_MESSAGE  
    );  
}
```

```
private Boolean obterMaoDeObraInformada() {  
    if (maoDeObraSimRadioButton.isSelected()) return true;  
    if (maoDeObraNaoRadioButton.isSelected()) return false;  
    return null;  
}
```

```
private void definirMaoDeObra(Boolean valor) {  
    maoDeObraButtonGroup.clearSelection();  
    if (valor == null) return;  
  
    if (valor) {  
        maoDeObraSimRadioButton.setSelected(true);  
    } else {  
        maoDeObraNaoRadioButton.setSelected(true);  
    }  
}
```

```
private Pecas obterPecasInformada() {  
    String codigoStr = codigoTextField.getText();  
    if (codigoStr.isEmpty()) {  
        throw new IllegalArgumentException("Informe o código da peça.");  
    }  
}
```

```
int codigo;  
try {  
    codigo = Integer.parseInt(codigoStr);  
} catch (NumberFormatException e) {  
    throw new IllegalArgumentException("Código da peça inválido.");  
}
```

```
}
```

```
String nome = nomeTextField.getText();
```

```
if (nome.isEmpty()) {
```

```
    throw new IllegalArgumentException("Informe o nome da peça.");
```

```
}
```

```
Pecas.MarcaPeca marca =
```

```
    (Pecas.MarcaPeca) marcaComboBox.getSelectedItem();
```

```
if (marca == null) {
```

```
    throw new IllegalArgumentException("Informe a marca da peça.");
```

```
}
```

```
String precoStr = precoTextField.getText();
```

```
if (precoStr.isEmpty()) {
```

```
    throw new IllegalArgumentException("Informe o preço da peça.");
```

```
}
```

```
double preco;
```

```
try {
```

```
    preco = Double.parseDouble(precoStr.replace(",", "."));
```

```
} catch (NumberFormatException e) {
```

```
    throw new IllegalArgumentException("Preço da peça inválido.");
```

```
}
```

```
String cor = corTextField.getText();
```

```
String prazoGarantiaStr = prazoGarantiaTextField.getText();
```

```
Integer prazoGarantia = null;
```

```
if (!prazoGarantiaStr.isEmpty()) {
```

```
    try {
```

```
        prazoGarantia = Integer.parseInt(prazoGarantiaStr);
```

```
    } catch (NumberFormatException e) {
```

```
        throw new IllegalArgumentException("Prazo de garantia inválido.");
```

```
    }
```

```
}
```

```
Boolean mao_de_obra = obterMaoDeObraInformada();
```

```

        if (mao_de_obra == null) {
            throw new IllegalArgumentException("Selecione se a peça possui mão de obra.");
        }

        return new Pecas(
            codigo, nome, marca, preco, mao_de_obra, prazoGarantia,
            cor.isEmpty() ? null : cor
        );
    }

```

```

private Pecas getVisaoAlterada(int codigo) {
    for (Pecas visao : pecas_cadastradas) {
        if (visao.getCodigo() == codigo) return visao;
    }
    return null;
}

```

```

@SuppressWarnings("unchecked")
// <editor-fold defaultstate="collapsed" desc="Generated Code">
private void initComponents() {
    java.awt.GridBagConstraints gridBagConstraints;

    comandosPanel = new javax.swing.JPanel();
    inserirPecas = new javax.swing.JButton();
    alterarPecas = new javax.swing.JButton();
    consultarPecas = new javax.swing.JButton();
    removerPecas = new javax.swing.JButton();
    limparCampos = new javax.swing.JButton();
    codigoLabel = new javax.swing.JLabel();
    codigoTextField = new javax.swing.JTextField();
    nomeLabel = new javax.swing.JLabel();
    nomeTextField = new javax.swing.JTextField();
    categoriaLabel = new javax.swing.JLabel();
    categoriaTextField = new javax.swing.JTextField();
    precoLabel = new javax.swing.JLabel();
    precoTextField = new javax.swing.JTextField();
    tipoLabel = new javax.swing.JLabel();

```

```

tipoTextField = new javax.swing.JTextField();
corLabel = new javax.swing.JLabel();
corTextField = new javax.swing.JTextField();
maoDeObraLabel = new javax.swing.JLabel();
maoDeObraPanel = new javax.swing.JPanel();
maoDeObraSimRadioButton = new javax.swing.JRadioButton();
maoDeObraNaoRadioButton = new javax.swing.JRadioButton();
pecas_cadastradasComboBox = new javax.swing.JComboBox();
pecasCadastradasLabel = new javax.swing.JLabel();

setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);
setTitle("Cadastrar Peças");
setMinimumSize(new java.awt.Dimension(640, 420));
setPreferredSize(new java.awt.Dimension(720, 560));
java.awt.GridBagLayout layout = new java.awt.GridBagLayout();
layout.rowHeights = new int[] {8};
getContentPane().setLayout(layout);

inserirPecas.setText("Inserir");
inserirPecas.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        inserirPecas(evt);
    }
});
comandosPanel.add(inserirPecas);

alterarPecas.setText("Alterar");
alterarPecas.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        alterarPecas(evt);
    }
});
comandosPanel.add(alterarPecas);

consultarPecas.setText("Consultar");
consultarPecas.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {

```

```
        consultarPecas(evt);
    }
});
comandosPanel.add(consultarPecas);
```

```
removerPecas.setText("Remover");
removerPecas.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        removerPecas(evt);
    }
});
comandosPanel.add(removerPecas);
```

```
limparCampos.setText("Limpar");
limparCampos.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        limparCampos(evt);
    }
});
comandosPanel.add(limparCampos);
```

```
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 9;
gridBagConstraints.insets = new java.awt.Insets(5, 5, 5, 5);
getContentPane().add(comandosPanel, gridBagConstraints);
```

```
codigoLabel.setText("Código");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 1;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(codigoLabel, gridBagConstraints);
```

```
codigoTextField.setColumns(50);
codigoTextField.setPreferredSize(new java.awt.Dimension(80, 20));
```

```
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 1;
gridBagConstraints.ipadx = 399;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(codigoTextField, gridBagConstraints);
```

```
nomeLabel.setText("Nome");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 2;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(nomeLabel, gridBagConstraints);
```

```
nomeTextField.setColumns(50);
nomeTextField.setPreferredSize(new java.awt.Dimension(240, 20));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 2;
gridBagConstraints.ipadx = 200;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(nomeTextField, gridBagConstraints);
```

```
categoriaLabel.setText("Categoria");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 3;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(categoriaLabel, gridBagConstraints);
```

```
categoriaTextField.setColumns(50);
categoriaTextField.setPreferredSize(new java.awt.Dimension(180, 20));
gridBagConstraints = new java.awt.GridBagConstraints();
```

```
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 3;
gridBagConstraints.ipadx = 100;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(categoriaTextField, gridBagConstraints);
```

```
precoLabel.setText("Preço");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 4;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(precoLabel, gridBagConstraints);
```

```
precoTextField.setColumns(50);
precoTextField.setPreferredSize(new java.awt.Dimension(100, 20));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 4;
gridBagConstraints.ipadx = 18;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(precoTextField, gridBagConstraints);
```

```
tipoLabel.setText("Tipo");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 5;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(tipoLabel, gridBagConstraints);
```

```
tipoTextField.setColumns(14);
tipoTextField.setPreferredSize(new java.awt.Dimension(140, 20));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
```

```
gridBagConstraints.gridy = 5;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(tipoTextField, gridBagConstraints);
```

```
corLabel.setText("Cor");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 6;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(corLabel, gridBagConstraints);
```

```
corTextField.setColumns(10);
corTextField.setPreferredSize(new java.awt.Dimension(100, 20));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 6;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(corTextField, gridBagConstraints);
```

```
maoDeObraLabel.setText("Mão de Obra");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 7;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(maoDeObraLabel, gridBagConstraints);
```

```
maoDeObraPanel.setLayout(new java.awt.FlowLayout(java.awt.FlowLayout.LEFT));
```

```
maoDeObraSimRadioButton.setText("Sim");
maoDeObraPanel.add(maoDeObraSimRadioButton);
```

```
maoDeObraNaoRadioButton.setText("Não");
maoDeObraPanel.add(maoDeObraNaoRadioButton);
```



```
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 7;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(maoDeObraPanel, gridBagConstraints);
```

```
pecas_cadastradasComboBox.setModel(new DefaultComboBoxModel(pecas_cadastradas));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 0;
gridBagConstraints.anchor = java.awt.GridBagConstraints.NORTHWEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(pecas_cadastradasComboBox, gridBagConstraints);
```

```
pecasCadastradasLabel.setText("Peças Cadastradas");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 0;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(pecasCadastradasLabel, gridBagConstraints);
```

```
pack();
} // </editor-fold>
```

```
private void inserirPecas(java.awt.event.ActionEvent evt) {
    Pecas pecas;
    String mensagem_erro = null;

    try {
        pecas = obterPecasInformada();
    } catch (IllegalArgumentException excecao) {
        informarErro(excecao.getMessage());
        return;
    }
}
```

```

mensagem_erro = controlador.inserirPecas(pecas);

if (mensagem_erro == null) {
    Pecas visao = pecas.getVisao();
    atualizarPecasCadastradas(visao.getCodigo());
} else {
    informarErro(mensagem_erro);
}
}

private void alterarPecas(java.awt.event.ActionEvent evt) {
    Pecas pecas;
    String mensagem_erro = null;

    try {
        pecas = obterPecasInformada();
    } catch (IllegalArgumentException excecao) {
        informarErro(excecao.getMessage());
        return;
    }

    mensagem_erro = controlador.alterarPecas(pecas);

    if (mensagem_erro == null) {
        Pecas visao = getVisaoAlterada(pecas.getCodigo());

        if (visao != null) {
            visao.setCodigo(pecas.getCodigo());
            visao.setNome(pecas.getNome());
            visao.setMarca(pecas.getMarca());
            visao.setPreco(pecas.getPreco());
            visao.setCor(pecas.getCor());
            visao.setMaoDeObra(pecas.getMaoDeObra());
            visao.setDiasGarantia(pecas.getDiasGarantia());

            pecas_cadastradasComboBox.updateUI();
        }
    }
}

```

```

    }
} else {
    informarErro(mensagem_erro);
}
}

```

```

private void consultarPecas(java.awt.event.ActionEvent evt) {
    Pecas visao =
        (Pecas) pecas_cadastradasComboBox.getSelectedItem();

    Pecas pecas = null;
    String mensagem_erro = null;

    if (visao != null) {
        pecas = Pecas.buscarPecas(visao.getCodigo());

        if (pecas == null)
            mensagem_erro = "Peça não cadastrada";
        } else {
            mensagem_erro = "Nenhuma Peça selecionada";
        }

    if (mensagem_erro == null) {

        codigoTextField.setText(
            String.valueOf(pecas.getCodigo())
        );

        nomeTextField.setText(pecas.getNome());
        marcaComboBox.setSelectedItem(pecas.getMarca());

        precoTextField.setText(
            String.valueOf(pecas.getPreco())
        );

        corTextField.setText(pecas.getCor() != null ? pecas.getCor() : "");
        prazoGarantiaTextField.setText(

```

```

        pecas.getDiasGarantia() != null
            ? String.valueOf(pecas.getDiasGarantia())
            : ""
    );

    definirMaoDeObra(pecas.getMaoDeObra());

} else {
    informarErro(mensagem_erro);
}
}

private void removerPecas(java.awt.event.ActionEvent evt) {
    Pecas visao =
        (Pecas) pecas_cadastradasComboBox.getSelectedItem();

    String mensagem_erro = null;

    if (visao != null)
        mensagem_erro = controlador.removerPecas(visao.getCodigo());
    else
        mensagem_erro = "Nenhuma Peça selecionada";

    if (mensagem_erro == null) {
        atualizarPecasCadastradas();
        limparCampos(evt);
    } else {
        informarErro(mensagem_erro);
    }
}

private void limparCampos(java.awt.event.ActionEvent evt) {
    codigoTextField.setText("");
    nomeTextField.setText("");
    marcaComboBox.setSelectedIndex(-1);
    precoTextField.setText("");
    corTextField.setText("");
}

```

```
    prazoGarantiaTextField.setText("");
    maoDeObraButtonGroup.clearSelection();
}
```

```
// Variables declaration - do not modify
```

```
private javax.swing.JButton alterarPecas;
private javax.swing.JLabel categoriaLabel;
private javax.swing.JTextField categoriaTextField;
private javax.swing.JLabel codigoLabel;
private javax.swing.JTextField codigoTextField;
private javax.swing.JPanel comandosPanel;
private javax.swing.JButton consultarPecas;
private javax.swing.JLabel corLabel;
private javax.swing.JTextField corTextField;
private javax.swing.JButton inserirPecas;
private javax.swing.JButton limparCampos;
private javax.swing.JLabel maoDeObraLabel;
private javax.swing.JRadioButton maoDeObraNaoRadioButton;
private javax.swing.JPanel maoDeObraPanel;
private javax.swing.JRadioButton maoDeObraSimRadioButton;
private javax.swing.JLabel nomeLabel;
private javax.swing.JTextField nomeTextField;
private javax.swing.JLabel pecasCadastradasLabel;
private javax.swing.JComboBox pecas_cadastradasComboBox;
private javax.swing.JLabel precoLabel;
private javax.swing.JTextField precoTextField;
private javax.swing.JButton removerPecas;
private javax.swing.JLabel tipoLabel;
private javax.swing.JTextField tipoTextField;
// End of variables declaration
```

```
}
```

\$\$\$ src.interfaces.JanelaCadastroSeguradoras

```
package interfaces;

import javax.swing.JOptionPane;
import javax.swing.ButtonGroup;
import controles.ControladorCadastroSeguradoras;
import entidades.Seguradora;
import javax.swing.DefaultComboBoxModel;
import java.awt.Frame;

public class JanelaCadastroSeguradoras extends javax.swing.JFrame {

    ControladorCadastroSeguradoras controlador;
    Seguradora[] seguradoras_cadastradas;

    public JanelaCadastroSeguradoras(ControladorCadastroSeguradoras controlador) {
        this(controlador, null);
    }

    public JanelaCadastroSeguradoras(
        ControladorCadastroSeguradoras controlador,
        Frame owner
    ) {
        this.controlador = controlador;
        seguradoras_cadastradas = Seguradora.getVisoes();
        initComponents();
        atualizarSeguradorasCadastradas();
        setSize(new java.awt.Dimension(760, 540));
        configurarJanelaDependente(owner);
        limparCampos(null);
    }

    private void configurarJanelaDependente(Frame owner) {
        setAutoRequestFocus(true);
        setAlwaysOnTop(true);
    }
}
```

```

    if (owner == null) {
        setLocationByPlatform(true);
        return;
    }

    posicionarRelativaAoOwner(owner);
}

private void posicionarRelativaAoOwner(Frame owner) {
    int x = owner.getX() + (owner.getWidth() - getWidth()) / 2;
    int y = owner.getY() + (owner.getHeight() - getHeight()) / 2;
    setLocation(x, y);
}

private Seguradora localizarSeguradora(String nome) {
    if (nome == null) return null;
    for (Seguradora visao : seguradoras_cadastradas) {
        if (nome.equals(visao.getNome())) return visao;
    }
    return null;
}

private void atualizarSeguradorasCadastradas() {
    atualizarSeguradorasCadastradas(null);
}

private void atualizarSeguradorasCadastradas(String nome_selecionado) {
    seguradoras_cadastradas = Seguradora.getVisoes();
    seguradoras_cadastradasComboBox.setModel(
        new DefaultComboBoxModel(seguradoras_cadastradas)
    );

    Seguradora visao_selecionada = localizarSeguradora(nome_selecionado);
    if (visao_selecionada != null) {
        seguradoras_cadastradasComboBox.setSelectedItem(visao_selecionada);
    }
}

```

```

private void informarErro(String mensagem) {
    JOptionPane.showMessageDialog(
        this,
        mensagem,
        "Erro",
        JOptionPane.ERROR_MESSAGE
    );
}

```

```

private Boolean obterPossuiAtendimento24hInformado() {
    if (possuiAtendimento24hSimRadioButton.isSelected()) return true;
    if (possuiAtendimento24hNaoRadioButton.isSelected()) return false;
    return null;
}

```

```

private void definirPossuiAtendimento24h(Boolean valor) {
    possuiAtendimento24hButtonGroup.clearSelection();
    if (valor == null) return;

    if (valor) {
        possuiAtendimento24hSimRadioButton.setSelected(true);
    } else {
        possuiAtendimento24hNaoRadioButton.setSelected(true);
    }
}

```

```

private Seguradora.FormaPagamentoPreferencial
obterFormaPagamentoPreferencialSelecionada() {
    Object selecionado = formaPagamentoPreferencialComboBox.getSelectedItem();
    if (selecionado == null) {
        return null;
    }

    return Seguradora.FormaPagamentoPreferencial.fromTexto(selecionado.toString());
}

```



```
private Seguradora obterSeguradoraInformada() {

    String nome = nomeTextField.getText();
    if (nome.isEmpty()) return null;

    String cidade = cidadeTextField.getText();
    if (cidade.isEmpty()) cidade = null;

    String cobertura_str = coberturaPercentualTextField.getText();
    if (cobertura_str.isEmpty()) return null;

    double cobertura_percentual;

    try {
        cobertura_percentual =
            Double.parseDouble(cobertura_str.replace(",", "."));
    } catch (NumberFormatException e) {
        return null;
    }

    Boolean possui_atendimento_24h =
        obterPossuiAtendimento24hInformado();
    if (possui_atendimento_24h == null) return null;

    Seguradora.FormaPagamentoPreferencial forma_pagamento_preferencial =
        obterFormaPagamentoPreferencialSelecionada();
    if (forma_pagamento_preferencial == null) return null;

    return new Seguradora(
        nome,
        cidade,
        cobertura_percentual,
        possui_atendimento_24h,
        forma_pagamento_preferencial
    );
}
```

```

private Seguradora getVisaoAlterada(String nome) {
    for (Seguradora visao : seguradoras_cadastradas) {
        if (visao.getNome().equals(nome)) return visao;
    }
    return null;
}

```

```

@SuppressWarnings("unchecked")

```

```

// <editor-fold defaultstate="collapsed" desc="Generated Code">

```

```

private void initComponents() {
    java.awt.GridBagConstraints gridBagConstraints;

    comandosPanel = new javax.swing.JPanel();
    inserirSeguradora = new javax.swing.JButton();
    alterarSeguradora = new javax.swing.JButton();
    consultarSeguradora = new javax.swing.JButton();
    removerSeguradora = new javax.swing.JButton();
    limparCampos = new javax.swing.JButton();
    nomeLabel = new javax.swing.JLabel();
    nomeTextField = new javax.swing.JTextField();
    cidadeLabel = new javax.swing.JLabel();
    cidadeTextField = new javax.swing.JTextField();
    coberturaPercentualLabel = new javax.swing.JLabel();
    coberturaPercentualTextField = new javax.swing.JTextField();
    possuiAtendimento24hLabel = new javax.swing.JLabel();
    possuiAtendimento24hPanel = new javax.swing.JPanel();
    possuiAtendimento24hSimRadioButton = new javax.swing.JRadioButton();
    possuiAtendimento24hNaoRadioButton = new javax.swing.JRadioButton();
    seguradorasCadastradasLabel = new javax.swing.JLabel();
    seguradoras_cadastradasComboBox = new javax.swing.JComboBox();

    setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);
    setTitle("Cadastrar Seguradoras");
    setMinimumSize(new java.awt.Dimension(620, 360));
    setPreferredSize(new java.awt.Dimension(700, 460));
    getContentPane().setLayout(new java.awt.GridBagLayout());
}

```

```
inserirSeguradora.setText("Inserir");
inserirSeguradora.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        inserirSeguradora(evt);
    }
});
comandosPanel.add(inserirSeguradora);
```

```
alterarSeguradora.setText("Alterar");
alterarSeguradora.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        alterarSeguradora(evt);
    }
});
comandosPanel.add(alterarSeguradora);
```

```
consultarSeguradora.setText("Consultar");
consultarSeguradora.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        consultarSeguradora(evt);
    }
});
comandosPanel.add(consultarSeguradora);
```

```
removerSeguradora.setText("Remover");
removerSeguradora.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        removerSeguradora(evt);
    }
});
comandosPanel.add(removerSeguradora);
```

```
limparCampos.setText("Limpar");
limparCampos.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        limparCampos(evt);
    }
});
```

```
});
```

```
comandosPanel.add(limparCampos);
```

```
gridBagConstraints = new java.awt.GridBagConstraints();
```

```
gridBagConstraints.gridx = 1;
```

```
gridBagConstraints.gridy = 5;
```

```
gridBagConstraints.insets = new java.awt.Insets(5, 5, 5, 5);
```

```
getContentPane().add(comandosPanel, gridBagConstraints);
```

```
nomeLabel.setText("Nome");
```

```
nomeLabel.setMaximumSize(null);
```

```
nomeLabel.setMinimumSize(null);
```

```
gridBagConstraints = new java.awt.GridBagConstraints();
```

```
gridBagConstraints.gridx = 0;
```

```
gridBagConstraints.gridy = 1;
```

```
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
```

```
gridBagConstraints.insets = new java.awt.Insets(1, 16, 10, 10);
```

```
getContentPane().add(nomeLabel, gridBagConstraints);
```

```
nomeTextField.setColumns(50);
```

```
nomeTextField.setPreferredSize(new java.awt.Dimension(240, 20));
```

```
gridBagConstraints = new java.awt.GridBagConstraints();
```

```
gridBagConstraints.gridx = 1;
```

```
gridBagConstraints.gridy = 1;
```

```
gridBagConstraints.ipadx = 80;
```

```
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
```

```
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
```

```
getContentPane().add(nomeTextField, gridBagConstraints);
```

```
cidadeLabel.setText("Cidade");
```

```
gridBagConstraints = new java.awt.GridBagConstraints();
```

```
gridBagConstraints.gridx = 0;
```

```
gridBagConstraints.gridy = 2;
```

```
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
```

```
gridBagConstraints.insets = new java.awt.Insets(1, 16, 10, 10);
```

```
getContentPane().add(cidadeLabel, gridBagConstraints);
```

```
cidadeTextField.setColumns(50);
cidadeTextField.setPreferredSize(new java.awt.Dimension(180, 20));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 2;
gridBagConstraints.ipadx = 20;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(cidadeTextField, gridBagConstraints);
```

```
coberturaPercentualLabel.setText("Cobertura Percentual");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 3;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 16, 10, 10);
getContentPane().add(coberturaPercentualLabel, gridBagConstraints);
```

```
coberturaPercentualTextField.setColumns(50);
coberturaPercentualTextField.setPreferredSize(new java.awt.Dimension(100, 20));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 3;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(coberturaPercentualTextField, gridBagConstraints);
```

```
possuiAtendimento24hLabel.setText("Possui Atendimento 24h");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 4;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 16, 10, 10);
getContentPane().add(possuiAtendimento24hLabel, gridBagConstraints);
```

```
possuiAtendimento24hPanel.setLayout(new
java.awt.FlowLayout(java.awt.FlowLayout.LEFT));
```

```
possuiAtendimento24hSimRadioButton.setText("Sim");
possuiAtendimento24hPanel.add(possuiAtendimento24hSimRadioButton);
```

```
possuiAtendimento24hNaoRadioButton.setText("Não");
possuiAtendimento24hPanel.add(possuiAtendimento24hNaoRadioButton);
```

```
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 4;
gridBagConstraints.gridwidth = 2;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(possuiAtendimento24hPanel, gridBagConstraints);
```

```
seguradorasCadastradasLabel.setText("Seguradoras Cadastradas");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(0, 12, 0, 5);
getContentPane().add(seguradorasCadastradasLabel, gridBagConstraints);
```

```
seguradoras_cadastradasComboBox.setModel(new
DefaultComboBoxModel(seguradoras_cadastradas));
seguradoras_cadastradasComboBox.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        seguradoras_cadastradasComboBoxActionPerformed(evt);
    }
});
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 0;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(seguradoras_cadastradasComboBox, gridBagConstraints);

pack();
} // </editor-fold>
```

```
private void alterarSeguradora(java.awt.event.ActionEvent evt) {  
    Seguradora seguradora =  
        obterSeguradoraInformada();  
  
    String mensagem_erro = null;  
  
    if (seguradora != null)  
        mensagem_erro =  
            controlador.alterarSeguradora(seguradora);  
    else  
        mensagem_erro =  
            "Algum atributo da Seguradora não foi informado.";  
  
    if (mensagem_erro == null) {  
  
        Seguradora visao =  
            getVisaoAlterada(seguradora.getNome());  
  
        if (visao != null) {  
  
            visao.setCidade(  
                seguradora.getCidade()  
            );  
  
            visao.setCoberturaPercentual(  
                seguradora.getCoberturaPercentual()  
            );  
  
            visao.setPossuiAtendimento24h(  
                seguradora.getPossuiAtendimento24h()  
            );  
  
            visao.setFormaPagamentoPreferencial(  
                seguradora.getFormaPagamentoPreferencial()  
            );  
        }  
    }  
}
```

```

        seguradoras_cadastradasComboBox.updateUI();

        seguradoras_cadastradasComboBox
            .setSelectedItem(visao);
    }

} else {
    informarErro(mensagem_erro);
}
}

private void inserirSeguradora(java.awt.event.ActionEvent evt) {
    Seguradora seguradora =
        obterSeguradoraInformada();

    String mensagem_erro = null;

    if (seguradora != null)
        mensagem_erro =
            controlador.inserirSeguradora(seguradora);
    else
        mensagem_erro =
            "Algum atributo da Seguradora não foi informado";

    if (mensagem_erro == null) {

        Seguradora visao = seguradora.getVisao();
        atualizarSeguradorasCadastradas(visao.getNome());

    } else {
        informarErro(mensagem_erro);
    }
}

private void consultarSeguradora(java.awt.event.ActionEvent evt) {
    Seguradora visao =
        (Seguradora)

```



```
seguradoras_cadastradasComboBox
    .getSelectedItem();

Seguradora seguradora = null;

String mensagem_erro = null;

if (visao != null) {

    seguradora =
        Seguradora.buscarSeguradora(
            visao.getNome()
        );

    if (seguradora == null)
        mensagem_erro =
            "Seguradora não cadastrada";

} else {
    mensagem_erro =
        "Nenhuma Seguradora selecionada";
}

if (mensagem_erro == null) {

    nomeTextField.setText(
        seguradora.getNome()
    );

    String cidade =
        seguradora.getCidade();

    if (cidade == null)
        cidade = "";

    cidadeTextField.setText(cidade);
```

```

coberturaPercentualTextField.setText(
    String.valueOf(
        seguradora
            .getCoberturaPercentual()
    )
);

definirPossuiAtendimento24h(seguradora.getPossuiAtendimento24h());

formaPagamentoPreferencialComboBox.setSelectedItem(
    seguradora.getFormaPagamentoPreferencial().toString()
);

} else {
    informarErro(mensagem_erro);
}
}

private void removerSeguradora(java.awt.event.ActionEvent evt) {
    Seguradora visao =
        (Seguradora)
        seguradoras_cadastradasComboBox
            .getSelectedItem();

    String mensagem_erro = null;

    if (visao != null)
        mensagem_erro =
            controlador.removerSeguradora(
                visao.getNome()
            );
    else
        mensagem_erro =
            "Nenhuma Seguradora selecionada";

    if (mensagem_erro == null) {

```

```
atualizarSeguradorasCadastradas();
```

```
limparCampos(evt);
```

```
} else {
```

```
    informarErro(mensagem_erro);
```

```
}
```

```
}
```

```
private void limparCampos(java.awt.event.ActionEvent evt) {
```

```
    nomeTextField.setText("");
```

```
    cidadeTextField.setText("");
```

```
    coberturaPercentualTextField.setText("");
```

```
    possuiAtendimento24hButtonGroup.clearSelection();
```

```
    formaPagamentoPreferencialComboBox.setSelectedIndex(-1);
```

```
}
```

```
private void seguradoras_cadastradasComboBoxActionPerformed(java.awt.event.ActionEvent  
evt) {
```

```
}
```

```
// Variables declaration - do not modify
```

```
private javax.swing.JButton alterarSeguradora;
```

```
private javax.swing.JLabel cidadeLabel;
```

```
private javax.swing.JTextField cidadeTextField;
```

```
private javax.swing.JLabel coberturaPercentualLabel;
```

```
private javax.swing.JTextField coberturaPercentualTextField;
```

```
private javax.swing.JPanel comandosPanel;
```

```
private javax.swing.JButton consultarSeguradora;
```

```
private javax.swing.JButton inserirSeguradora;
```

```
private javax.swing.JButton limparCampos;
```

```
private javax.swing.JLabel nomeLabel;
```

```
private javax.swing.JTextField nomeTextField;
```

```
private javax.swing.JLabel possuiAtendimento24hLabel;
```

```
private javax.swing.JRadioButton possuiAtendimento24hNaoRadioButton;
```

```
private javax.swing.JPanel possuiAtendimento24hPanel;  
private javax.swing.JRadioButton possuiAtendimento24hSimRadioButton;  
private javax.swing.JButton removerSeguradora;  
private javax.swing.JLabel seguradorasCadastradasLabel;  
private javax.swing.JComboBox seguradoras_cadastradasComboBox;  
// End of variables declaration  
}
```

\$\$\$ src.interfaces.JanelaCadastroSinistros

```
package interfaces;

import controles.ControladorCadastroPecasSinistros;
import controles.ControladorCadastroSinistros;
import entidades.Sinistro;
import java.awt.Frame;
import javax.swing.DefaultComboBoxModel;
import javax.swing.DefaultListModel;
import javax.swing.JOptionPane;

public class JanelaCadastroSinistros extends javax.swing.JFrame {

    ControladorCadastroSinistros controlador;
    Sinistro[] sinistros_cadastrados;
    private final DefaultListModel pecasSinistroModel = new DefaultListModel();

    public JanelaCadastroSinistros(ControladorCadastroSinistros controlador) {
        this(controlador, null);
    }

    public JanelaCadastroSinistros(ControladorCadastroSinistros controlador, Frame owner) {
        this.controlador = controlador;
        sinistros_cadastrados = Sinistro.getVisoes();
        initComponents();
        atualizarSinistrosCadastrados();
        setSize(new java.awt.Dimension(820, 620));
        configurarJanelaDependente(owner);
        limparCampos(null);
    }

    private void configurarJanelaDependente(Frame owner) {
        setAutoRequestFocus(true);
        setAlwaysOnTop(true);

        if (owner == null) {
```

```
        setLocationByPlatform(true);  
        return;  
    }
```

```
    int x = owner.getX() + (owner.getWidth() - getWidth()) / 2;  
    int y = owner.getY() + (owner.getHeight() - getHeight()) / 2;  
    setLocation(x, y);  
}
```

```
private Sinistro localizarSinistro(int id) {  
    if (sinistros_cadastrados == null) return null;  
    for (Sinistro visao : sinistros_cadastrados) {  
        if (visao.getId() == id) return visao;  
    }  
    return null;  
}
```

```
private void atualizarSinistrosCadastrados() {  
    atualizarSinistrosCadastrados(0);  
}
```

```
private void atualizarSinistrosCadastrados(int idSelecionado) {  
    sinistros_cadastrados = Sinistro.getVisoes();  
    sinistros_cadastradosComboBox.setModel(new  
DefaultComboBoxModel(sinistros_cadastrados));
```

```
    Sinistro visao_selecionada = localizarSinistro(idSelecionado);  
    if (visao_selecionada != null) {  
        sinistros_cadastradosComboBox.setSelectedItem(visao_selecionada);  
    }  
}
```

```
private void informarErro(String mensagem) {  
    JOptionPane.showMessageDialog(this, mensagem, "Erro",  
JOptionPane.ERROR_MESSAGE);  
}
```

```

public void atualizarListaPecasSinistro(int sinistroId) {
    pecasSinistroModel.clear();
    if (sinistroId <= 0) {
        pecasSinistroList.setModel(pecasSinistroModel);
        return;
    }

    for (entidades.Pecas peca : entidades.Pecas.buscarPecasPorSinistro(sinistroId)) {
        pecasSinistroModel.addElement(peca);
    }
    pecasSinistroList.setModel(pecasSinistroModel);
}

private Boolean obterPerdaTotalInformada() {
    return perdaTotalCheckBox.isSelected();
}

private void definirPerdaTotal(Boolean valor) {
    perdaTotalCheckBox.setSelected(Boolean.TRUE.equals(valor));
}

private Sinistro obterSinistroInformado() {
    String segurado = seguradoTextField.getText().trim();
    if (segurado.isEmpty()) return null;

    String telefone = telefoneTextField.getText().trim();
    if (telefone.isEmpty()) return null;

    String cidade = clienteTextField.getText().trim();
    if (cidade.isEmpty()) cidade = null;

    Sinistro.GrauMonta grau_monta =
        (Sinistro.GrauMonta) grauMontaComboBox.getSelectedItem();
    if (grau_monta == null) return null;

    Boolean perda_total = obterPerdaTotalInformada();
    if (perda_total == null) return null;
}

```

```

int id = numeroTextField.getText().trim().isEmpty() ? 0
    : Integer.parseInt(numeroTextField.getText().trim());
return new Sinistro(id, segurado, telefone, cidade, grau_monta, perda_total);
}

```

```

private Sinistro getVisaoAlterada(int id) {
    for (Sinistro visao : sinistros_cadastrados) {
        if (visao.getId() == id) return visao;
    }
    return null;
}

```

```

@SuppressWarnings("unchecked")
// <editor-fold defaultstate="collapsed" desc="Generated Code">

```

```

private void initComponents() {
    java.awt.GridBagConstraints gridBagConstraints;

    comandosPanel = new javax.swing.JPanel();
    inserirSinistro = new javax.swing.JButton();
    alterarSinistro = new javax.swing.JButton();
    consultarSinistro = new javax.swing.JButton();
    removerSinistro = new javax.swing.JButton();
    pecasSinistro = new javax.swing.JButton();
    limparCampos = new javax.swing.JButton();
    sinistrosCadastradosLabel = new javax.swing.JLabel();
    sinistros_cadastradosComboBox = new javax.swing.JComboBox();
    numeroLabel = new javax.swing.JLabel();
    numeroTextField = new javax.swing.JTextField();
    seguradoLabel = new javax.swing.JLabel();
    seguradoTextField = new javax.swing.JTextField();
    clienteLabel = new javax.swing.JLabel();
    clienteTextField = new javax.swing.JTextField();
    telefoneLabel = new javax.swing.JLabel();
    telefoneTextField = new javax.swing.JTextField();
    grauMontaLabel = new javax.swing.JLabel();
    grauMontaComboBox = new javax.swing.JComboBox();
}

```



```

perdaTotalLabel = new javax.swing.JLabel();
perdaTotalPanel = new javax.swing.JPanel();
perdaTotalCheckBox = new javax.swing.JCheckBox();
pecasSinistroLabel = new javax.swing.JLabel();
pecasSinistroScrollPane = new javax.swing.JScrollPane();
pecasSinistroList = new javax.swing.JList();

setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);
setTitle("Cadastrar Sinistros");
setMinimumSize(new java.awt.Dimension(680, 420));
setPreferredSize(new java.awt.Dimension(820, 620));
getContentPane().setLayout(new java.awt.GridBagLayout());

inserirSinistro.setText("Inserir");
inserirSinistro.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        inserirSinistro(evt);
    }
});
comandosPanel.add(inserirSinistro);

alterarSinistro.setText("Alterar");
alterarSinistro.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        alterarSinistro(evt);
    }
});
comandosPanel.add(alterarSinistro);

consultarSinistro.setText("Consultar");
consultarSinistro.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        consultarSinistro(evt);
    }
});
comandosPanel.add(consultarSinistro);

```

```
removerSinistro.setText("Remover");
removerSinistro.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        removerSinistro(evt);
    }
});
comandosPanel.add(removerSinistro);
```

```
pecasSinistro.setText("Peças");
pecasSinistro.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        pecasSinistro(evt);
    }
});
comandosPanel.add(pecasSinistro);
```

```
limparCampos.setText("Limpar");
limparCampos.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        limparCampos(evt);
    }
});
comandosPanel.add(limparCampos);
```

```
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 8;
gridBagConstraints.anchor = java.awt.GridBagConstraints.NORTHWEST;
gridBagConstraints.insets = new java.awt.Insets(5, 5, 5, 5);
getContentPane().add(comandosPanel, gridBagConstraints);
```

```
sinistrosCadastradosLabel.setText("Sinistros Cadastrados");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 0;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
```

```
getContentPane().add(sinistrosCadastradosLabel, gridBagConstraints);
```

```
sinistros_cadastradosComboBox.setModel(new  
DefaultComboBoxModel(sinistros_cadastrados));
```

```
sinistros_cadastradosComboBox.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        sinistros_cadastradosComboBoxActionPerformed(evt);  
    }  
});
```

```
gridBagConstraints = new java.awt.GridBagConstraints();  
gridBagConstraints.gridx = 1;  
gridBagConstraints.gridy = 0;  
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;  
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);  
getContentPane().add(sinistros_cadastradosComboBox, gridBagConstraints);
```

```
numeroLabel.setText("ID");  
gridBagConstraints = new java.awt.GridBagConstraints();  
gridBagConstraints.gridx = 0;  
gridBagConstraints.gridy = 1;  
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;  
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);  
getContentPane().add(numeroLabel, gridBagConstraints);
```

```
numeroTextField.setEditable(false);  
numeroTextField.setColumns(8);  
numeroTextField.setMinimumSize(new java.awt.Dimension(80, 20));  
numeroTextField.setPreferredSize(new java.awt.Dimension(80, 20));  
gridBagConstraints = new java.awt.GridBagConstraints();  
gridBagConstraints.gridx = 1;  
gridBagConstraints.gridy = 1;  
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;  
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);  
getContentPane().add(numeroTextField, gridBagConstraints);
```

```
seguradoLabel.setText("Segurado");  
gridBagConstraints = new java.awt.GridBagConstraints();
```

```
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 2;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(seguradoLabel, gridBagConstraints);
```

```
seguradoTextField.setColumns(24);
seguradoTextField.setMinimumSize(new java.awt.Dimension(240, 20));
seguradoTextField.setPreferredSize(new java.awt.Dimension(240, 20));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 2;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(seguradoTextField, gridBagConstraints);
```

```
clienteLabel.setText("Cidade");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 3;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(clienteLabel, gridBagConstraints);
```

```
clienteTextField.setColumns(16);
clienteTextField.setMinimumSize(new java.awt.Dimension(160, 20));
clienteTextField.setPreferredSize(new java.awt.Dimension(160, 20));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 3;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(clienteTextField, gridBagConstraints);
```

```
telefoneLabel.setText("Telefone");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
```

```
gridBagConstraints.gridy = 4;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(telefoneLabel, gridBagConstraints);
```

```
telefoneTextField.setColumns(12);
telefoneTextField.setMinimumSize(new java.awt.Dimension(120, 20));
telefoneTextField.setPreferredSize(new java.awt.Dimension(120, 20));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 4;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(telefoneTextField, gridBagConstraints);
```

```
grauMontaLabel.setText("Grau de Monta");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 5;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(grauMontaLabel, gridBagConstraints);
```

```
grauMontaComboBox.setModel(new
DefaultComboBoxModel(Sinistro.GrauMonta.values()));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 5;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(grauMontaComboBox, gridBagConstraints);
```

```
perdaTotalLabel.setText("Perda Total");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 6;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
```

```
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);  
getContentPane().add(perdaTotalLabel, gridBagConstraintss);
```

```
perdaTotalPanel.setLayout(new java.awt.FlowLayout(java.awt.FlowLayout.LEFT));
```

```
perdaTotalCheckBox.setText("Sim");  
perdaTotalPanel.add(perdaTotalCheckBox);
```

```
gridBagConstraints = new java.awt.GridBagConstraints();  
gridBagConstraints.gridx = 1;  
gridBagConstraints.gridy = 6;  
gridBagConstraints.gridwidth = 2;  
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;  
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);  
getContentPane().add(perdaTotalPanel, gridBagConstraintss);
```

```
pecasSinistroLabel.setText("Peças do Sinistro");  
gridBagConstraints = new java.awt.GridBagConstraints();  
gridBagConstraints.gridx = 0;  
gridBagConstraints.gridy = 7;  
gridBagConstraints.anchor = java.awt.GridBagConstraints.NORTHEAST;  
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);  
getContentPane().add(pecasSinistroLabel, gridBagConstraintss);
```

```
pecasSinistroScrollPane.setPreferredSize(new java.awt.Dimension(300, 90));
```

```
pecasSinistroList.setSelectionMode(javax.swing.ListSelectionModel.SINGLE_SELECTION);  
pecasSinistroList.setModel(pecasSinistroModel);  
pecasSinistroScrollPane.setViewportView(pecasSinistroList);
```

```
gridBagConstraints = new java.awt.GridBagConstraints();  
gridBagConstraints.gridx = 1;  
gridBagConstraints.gridy = 7;  
gridBagConstraints.fill = java.awt.GridBagConstraints.BOTH;  
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;  
gridBagConstraints.weightx = 1.0;  
gridBagConstraints.weighty = 1.0;
```

```

gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(pecasSinistroScrollPane, gridBagConstraints);

pack();
} // </editor-fold>

private void sinistros_cadastradosComboBoxActionPerformed(java.awt.event.ActionEvent evt) {
    Sinistro visao = (Sinistro) sinistros_cadastradosComboBox.getSelectedItem();
    atualizarListaPecasSinistro(visao != null ? visao.getId() : 0);
}

private void limparCampos(java.awt.event.ActionEvent evt) {
    numeroTextField.setText("");
    seguradoTextField.setText("");
    clienteTextField.setText("");
    telefoneTextField.setText("");
    grauMontaComboBox.setSelectedIndex(-1);
    perdaTotalCheckBox.setSelected(false);
    atualizarListaPecasSinistro(0);
}

private void consultarSinistro(java.awt.event.ActionEvent evt) {
    Sinistro visao = (Sinistro) sinistros_cadastradosComboBox.getSelectedItem();
    String mensagem_erro = null;
    if (visao == null) {
        mensagem_erro = "Nenhum Sinistro selecionado";
    } else {
        Sinistro sinistro = Sinistro.buscarSinistro(visao.getId());
        if (sinistro == null) {
            mensagem_erro = "Sinistro não cadastrado";
        } else {
            numeroTextField.setText(String.valueOf(sinistro.getId()));
            seguradoTextField.setText(sinistro.getSegurado());
            clienteTextField.setText(sinistro.getCidade());
            telefoneTextField.setText(sinistro.getTelefone());
            grauMontaComboBox.setSelectedItem(sinistro.getGrauMonta());
            definirPerdaTotal(sinistro.getPerdaTotal());
        }
    }
}

```

```

        atualizarListaPecasSinistro(sinistro.getId());
    }
}

if (mensagem_erro != null) informarErro(mensagem_erro);
}

private void inserirSinistro(java.awt.event.ActionEvent evt) {
    Sinistro sinistro = obterSinistroInformado();
    String mensagem_erro = null;

    if (sinistro != null) {
        mensagem_erro = controlador.inserirSinistro(sinistro);
    } else {
        mensagem_erro = "Algum atributo do Sinistro não foi informado";
    }

    if (mensagem_erro == null) {
        atualizarSinistrosCadastrados(sinistro.getId());
        atualizarListaPecasSinistro(sinistro.getId());
        numeroTextField.setText(String.valueOf(sinistro.getId()));
        sinistros_cadastradosComboBox.setSelectedItem(getVisaoAlterada(sinistro.getId()));
    } else {
        informarErro(mensagem_erro);
    }
}

private void alterarSinistro(java.awt.event.ActionEvent evt) {
    Sinistro sinistro = obterSinistroInformado();
    String mensagem_erro = null;

    if (sinistro != null) {
        mensagem_erro = controlador.alterarSinistro(sinistro);
    } else {
        mensagem_erro = "Algum atributo do Sinistro não foi informado";
    }
}

```



```

if (mensagem_erro == null) {
    atualizarSinistrosCadastrados(sinistro.getId());
    atualizarListaPecasSinistro(sinistro.getId());
    sinistros_cadastradosComboBox.setSelectedItem(getVisaoAlterada(sinistro.getId()));
} else {
    informarErro(mensagem_erro);
}
}

```

```

private void removerSinistro(java.awt.event.ActionEvent evt) {
    Sinistro visao = (Sinistro) sinistros_cadastradosComboBox.getSelectedItem();
    String mensagem_erro = null;

    if (visao != null) {
        mensagem_erro = controlador.removerSinistro(visao.getId());
    } else {
        mensagem_erro = "Nenhum Sinistro selecionado";
    }

    if (mensagem_erro == null) {
        atualizarSinistrosCadastrados();
        limparCampos(evt);
    } else {
        informarErro(mensagem_erro);
    }
}

```

```

private void pecasSinistro(java.awt.event.ActionEvent evt) {
    Sinistro visao = (Sinistro) sinistros_cadastradosComboBox.getSelectedItem();
    String mensagem_erro = null;

    if (visao == null) {
        mensagem_erro = "Nenhum Sinistro selecionado";
    } else {
        Sinistro sinistro = Sinistro.buscarSinistro(visao.getId());
        if (sinistro == null) {
            mensagem_erro = "Sinistro não cadastrado";
        }
    }
}

```

```

        } else {
            new ControladorCadastroPecasSinistros(this, sinistro);
            atualizarListaPecasSinistro(sinistro.getId());
        }
    }

    if (mensagem_erro != null) {
        informarErro(mensagem_erro);
    }
}

// Variables declaration - do not modify
private javax.swing.JButton alterarSinistro;
private javax.swing.JLabel clienteLabel;
private javax.swing.JTextField clienteTextField;
private javax.swing.JPanel comandosPanel;
private javax.swing.JButton consultarSinistro;
private javax.swing.JComboBox grauMontaComboBox;
private javax.swing.JLabel grauMontaLabel;
private javax.swing.JButton inserirSinistro;
private javax.swing.JButton limparCampos;
private javax.swing.JLabel numeroLabel;
private javax.swing.JTextField numeroTextField;
private javax.swing.JButton pecasSinistro;
private javax.swing.JLabel pecasSinistroLabel;
private javax.swing.JList pecasSinistroList;
private javax.swing.JScrollPane pecasSinistroScrollPane;
private javax.swing.JCheckBox perdaTotalCheckBox;
private javax.swing.JLabel perdaTotalLabel;
private javax.swing.JPanel perdaTotalPanel;
private javax.swing.JButton removerSinistro;
private javax.swing.JLabel seguroLabel;
private javax.swing.JTextField seguroTextField;
private javax.swing.JLabel sinistrosCadastradosLabel;
private javax.swing.JComboBox sinistros_cadastradosComboBox;
private javax.swing.JLabel telefoneLabel;
private javax.swing.JTextField telefoneTextField;

```

```
// End of variables declaration
```

```
}
```

\$\$\$ src.interfaces.JanelaSistema

```
package interfaces;

import controles.ControladorCadastroSeguradoras;
import controles.ControladorCadastroPecas;
import controles.ControladorCadastroSinistros;
import javax.swing.JOptionPane;
import persistência.BD;

public class JanelaSistema extends javax.swing.JFrame {

    public JanelaSistema() {
        BD.criaConexao();
        initComponents();
    }

    @SuppressWarnings("unchecked")

    private void informarServiçoIndisponível() {
        JOptionPane.showMessageDialog(
            this,
            "Serviço Indisponível",
            "Informação",
            JOptionPane.INFORMATION_MESSAGE
        );
    }

    // <editor-fold defaultstate="collapsed" desc="Generated Code">
    private void initComponents() {

        seguradoras_orçamentoMenuBar = new javax.swing.JMenuBar();
        peçaMenu = new javax.swing.JMenu();
        cadastrar_peçaMenuItem = new javax.swing.JMenuItem();
        seguradoraMenu = new javax.swing.JMenu();
        cadastrar_seguradoraItemMenu = new javax.swing.JMenuItem();
        sinistroMenu = new javax.swing.JMenu();
```

```

cadastrar_sinistroMenuItem = new javax.swing.JMenuItem();
orçamentoMenu = new javax.swing.JMenu();
cadastrar_orçamentoMenuItem = new javax.swing.JMenuItem();
pesquisar_orçamentoMenuItem = new javax.swing.JMenuItem();

setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);
setTitle("Orçamentos da Seguradora");
setAlwaysOnTop(true);
addWindowListener(new java.awt.event.WindowAdapter() {
    public void windowClosed(java.awt.event.WindowEvent evt) {
        terminarSistema(evt);
    }
});

peçaMenu.setText("Peça");

cadastrar_peçaMenuItem.setText("Cadastrar");
cadastrar_peçaMenuItem.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        cadastrarPeça(evt);
    }
});
peçaMenu.add(cadastrar_peçaMenuItem);

seguradoras_orçamentoMenuBar.add(peçaMenu);

seguradoraMenu.setText("Seguradora");

cadastrar_seguradoraItemMenu.setText("Cadastrar");
cadastrar_seguradoraItemMenu.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        cadastrarSeguradora(evt);
    }
});
seguradoraMenu.add(cadastrar_seguradoraItemMenu);

seguradoras_orçamentoMenuBar.add(seguradoraMenu);

```

```
sinistroMenu.setText("Sinistro");
```

```
cadastrar_sinistroMenuItem.setText("Cadastrar");
```

```
cadastrar_sinistroMenuItem.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        cadastrarSinistro(evt);  
    }  
});
```

```
sinistroMenu.add(cadastrar_sinistroMenuItem);
```

```
seguradoras_orçamentoMenuBar.add(sinistroMenu);
```

```
orçamentoMenu.setText("Orçamento");
```

```
cadastrar_orçamentoMenuItem.setText("Cadastrar");
```

```
cadastrar_orçamentoMenuItem.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        cadastrarOrçamentos(evt);  
    }  
});
```

```
orçamentoMenu.add(cadastrar_orçamentoMenuItem);
```

```
pesquisar_orçamentoMenuItem.setText("Pesquisar");
```

```
pesquisar_orçamentoMenuItem.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        pesquisarOrçamentos(evt);  
    }  
});
```

```
orçamentoMenu.add(pesquisar_orçamentoMenuItem);
```

```
seguradoras_orçamentoMenuBar.add(orçamentoMenu);
```

```
setJMenuBar(seguradoras_orçamentoMenuBar);
```

```
javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());  
getContentPane().setLayout(layout);
```

```
layout.setHorizontalGroup(  
    layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)  
        .addGap(0, 400, Short.MAX_VALUE)  
);  
layout.setVerticalGroup(  
    layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)  
        .addGap(0, 279, Short.MAX_VALUE)  
);  
  
pack();  
} // </editor-fold>
```

```
private void cadastrarSeguradora(java.awt.event.ActionEvent evt) {  
    new ControladorCadastroSeguradoras(this);  
}
```

```
private void terminarSistema(java.awt.event.WindowEvent evt) {  
    BD.fechaConexao();  
    System.exit(0);  
}
```

```
private void pesquisarOrçamentos(java.awt.event.ActionEvent evt) {  
    informarServiçoIndisponível();  
}
```

```
private void cadastrarPeça(java.awt.event.ActionEvent evt) {  
    new ControladorCadastroPecas(this);  
}
```

```
private void cadastrarOrçamentos(java.awt.event.ActionEvent evt) {  
    informarServiçoIndisponível();  
}
```

```
private void cadastrarSinistro(java.awt.event.ActionEvent evt) {  
    new ControladorCadastroSinistros(this);  
}
```

```
public static void main(String args[]) {  
    java.awt.EventQueue.invokeLater(new Runnable() {  
        public void run() {  
            new JanelaSistema().setVisible(true);  
        }  
    });  
}
```

```
// Variables declaration - do not modify
```

```
private javax.swing.JMenuItem cadastrar_orçamentoMenuItem;  
private javax.swing.JMenuItem cadastrar_peçaMenuItem;  
private javax.swing.JMenuItem cadastrar_seguradoraItemMenu;  
private javax.swing.JMenuItem cadastrar_sinistroMenuItem;  
private javax.swing.JMenu orçamentoMenu;  
private javax.swing.JMenuItem pesquisar_orçamentoMenuItem;  
private javax.swing.JMenu peçaMenu;  
private javax.swing.JMenu seguradoraMenu;  
private javax.swing.JMenuBar seguradoras_orçamentoMenuBar;  
private javax.swing.JMenu sinistroMenu;
```

```
// End of variables declaration
```

```
}
```


\$\$\$ src.persistência.BD

```
package persistência;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class BD {

    static final String URL_BD = "jdbc:mysql://localhost/banco";
    static final String USUARIO = "root";
    static final String SENHA = "admin";
    public static Connection conexao = null;

    public static void criaConexao() {
        try {
            conexao = DriverManager.getConnection(URL_BD, USUARIO, SENHA);
        } catch (SQLException excecao_sql) {
            excecao_sql.printStackTrace();
        }
    }

    public static void fechaConexao() {
        try {
            if (conexao != null) {
                conexao.close();
            }
        } catch (SQLException excecao_sql) {
            excecao_sql.printStackTrace();
        }
    }
}
```